Introduction

Contents

- 1. Garden Lattice
- 2. Software Forest
- 3. Verdant Sundial
- Seed Bank
- Checks Index
- Garden Circle



Welcome to the Software Gardening Almanack, an open-source handbook of applied guidance and tools for sustainable software development and maintenance.

The Software Gardening Almanack is for anyone who creates or maintains software. The Almanack instructs individuals - software developers, engineers, project leads, scientists, and beyond - on best nurturing practices to build software that stands the tests of time. If you've ever wondered why software grows fast only to decay just as quickly, or how to design systems that sustain long-term innovation, this is for you. This handbook provides pragmatic guidance and tools to cultivate resilient software practices. Whether you're tending legacy code or planting new ideas, ***jump in and start growing***! 🌱

Overview

The structure of the book includes the following *core chapters*:

1. The **Garden Lattice**: a chapter exploring the human-centric elements of software.
2. The **Software Forest**: a chapter examining code and its role in shaping software.
3. The **Verdant Sundial**: a chapter illustrating the evolution of people and code, and their impact on software sustainability.

In addition there are also *supplementary chapters* which help support or demonstrate the content found within the core chapters:

- The **Seed Bank**: an applied chapter which illustrates concepts from the Almanack core through Jupyter notebooks.

- The **Garden Circle**: a space where we openly document principles for developing and maintaining the book as well as document a related application programming interface (API) for the `almanack` Python package.

Using the Almanack

The Software Gardening Almanack project provides two primary components:

- The **Almanack handbook**: the content found here helps educate, demonstrate, and evolve the concepts of sustainable software development.
- The `almanack` **package**: is a Python package which implements the concepts of the book to help improve software sustainability by generating organized metrics and running linting checks on repositories.

The `almanack` package

Please see our (Seed Bank)[seed-bank-intro] **Software Gardening Almanack Example** for a quick demonstration of using the `almanack` Python package. The package may be installed using the following, for example:

```
# install from pypi
pip install almanack

# install directly from source
pip install git+https://github.com/software-gardening/almanack.git
```

Once installed, the Almanack can be used to analyze repositories for sustainable development practices. Output from the Almanack includes metrics which are defined through `metrics.yml` as a Python dictionary (JSON-compatible) record structure. Package API documentation may be found within the package API section of the book.

You can use the Almanack package as a command-line interface (CLI):

```
# generate a table of metrics based on a repository
almanack table path/to/repository

# perform linting-style checks on a repository
almanack check path/to/repository
```

We provide pre-commit hooks to enable you to run the Almanack as part of your automated checks for developing software projects. Add the following to your `pre-commit-config.yaml` in order to use the Almanack.

For example:

```
# include this in your pre-commit-config.yaml
- repo: https://github.com/software-gardening/almanack
    rev: v0.1.1
    hooks:
    - id: almanack-check
```

Inspiration

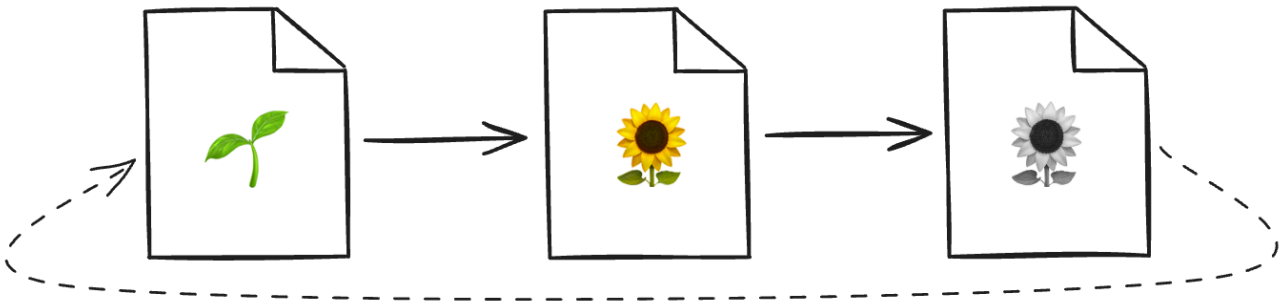


Fig. 1 Software is created, grows, and decays over time.

Software experiences development cycles, which accumulate errors over time. However, these cycles are not well understood nor are they explicitly cultivated with the impacts of time in mind. Why does software grow quickly only to decay just as fast? How do software bugs seem to appear in unlikely scenarios?

These cycles follow patterns from life: software is created, grows, decays, and so on (sometimes in surprising or seemingly unpredictable ways). Software is also connected within a complex system of relationships (similar to the complex ecology of a garden). The Software Gardening Almanack posits we can understand these software lifecycle patterns and complex relationships in order to build tools which sustain or maintain their development long-term.

> *"The 'planetary garden' is a means of considering ecology as the integration of humanity – the gardeners – into its smallest spaces. Its guiding philosophy is based on the principle of the 'garden in motion': do the most **for**, the minimum **against**."* - Gilles Clément

The content within the Software Gardening Almanack is inspired by ecological systems (for example, as in *planetary gardening*). We also are galvanized by the *scientific method*, and *almanacks* (or earlier *menologia rustica*) in documenting, expecting, and optimizing how we plan for time-based changes in agriculture (among other practices and traditions).

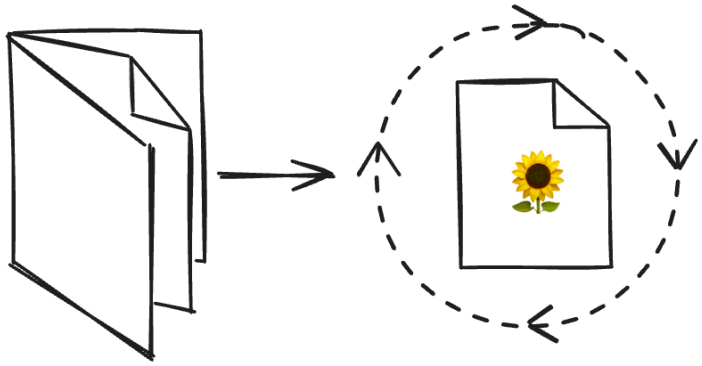


Fig. 2 Almanacks help us understand and influence the impacts of time on the things we grow.

The Software Gardening Almanack helps share knowledge on how to cultivate change in order to nurture software (and software practitioners) for long periods of time. We aspire to define, practice, and continually improve a craft of *software gardening* to nourish existing or yet-to-be software projects and embrace a more resilient future.

Motivation

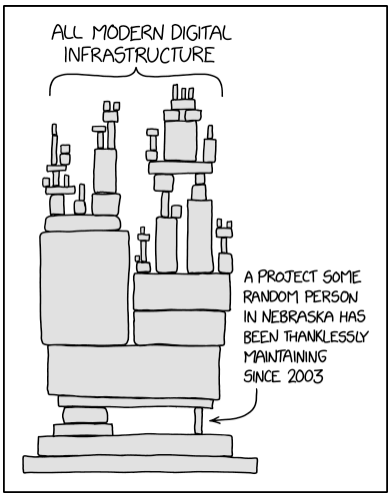


Fig. 3 We depend on a delicate network of software which changes over time. (Image source: 'Dependency' comic by Randall Munroe, XKCD)

Our world is surrounded by systems of software which enable and enhance life. These systems are both invisible and sometimes brittle, breaking in surprising ways (for example, beneath uneven pressures as in the above XKCD comic). At the same time, loose coupling of these software systems also forms the basis of innovation through multidimensional, emergent growth patterns. We seek to embrace these software systems which are innately in motion, embracing diversity without destroying it to perpetuate discovery and enhance future life.

> *"Start where you are. Use what you have. Do what you can."* - Arthur Ashe

We suggest reading or using this content however it best makes sense to a reader. Just as with gardening, sometimes the best thing to do is jump in! We structure each section in a modular fashion, providing insights with cited prerequisites. We provide links and other reference materials for further reading where beneficial. Please let us know how we can improve by opening GitHub issues or creating new discussion board posts (see our `contributing.md` guide for more information)!

Who's this for?

> *"We'll share stories from the heart. All are welcome here."* - Alexandra Penfold

The Software Gardening Almanack is designed for developers of any kind. When we say *developers* here we mean anyone engaging in pursuing their vision towards a goal using software (including scientists, engineers, project leads, managers, etc). The content will engage readers with pragmatic ideas, keeping the barrier to entry low.

Acknowledgements

This work was supported by the Better Scientific Software Fellowship Program, a collaborative effort of the U.S. Department of Energy (DOE), Office of Advanced Scientific Research via ANL under Contract DE-AC02-

06CH11357 and the National Nuclear Security Administration Advanced Simulation and Computing Program via LLNL under Contract DE-AC52-07NA27344; and by the National Science Foundation (NSF) via SHI under Grant No. 2327079.

Garden Lattice



Fig. 4 The garden lattice facilitates growth on our software gardening journeys. (Image source: GL1).

> “To plant a garden is to believe in tomorrow.” - Audrey Hepburn

We enter the book through a garden lattice, both an acknowledgement and forward recognition of people who cultivate software. The garden lattice is a celebration of our individual vibrancy and connection to the world around us. “Here you will give your gifts and meet your responsibilities.” GL2 It also is a place to view our collective fates; as we grow an understanding emerges that we must commit ourselves toward a sustainable future through tending the garden. This sustainability is undeniably coupled to our natural relationships, persisted even in the software realm.

Early roots from Ada Lovelace’s work on an analytical engine algorithm in 1842 provide a glimpse of the natural world’s influence on software development: “… the Analytical Engine weaves algebraical patterns just as the Jacquard-loom weaves flowers and leaves.” GL3 Many years later, software development remains an effort of intertwining beauty and function to enhance well-being through a growing system, or lattice, of interconnected parts built upon layers of personal and technical advances.

Lattices, reminiscent of living structures built with willow or bamboo, emerge as a weaved foundation, both flexible and systematically resilient, protecting and inspiring new growth. “The willow submits to the wind and prospers until one day it is many willows - a wall against the wind.” GL4. These lattices also signify our collective friendship and interdependence. They echo the sentiment of “Be like bamboo. The higher you grow, the deeper you bow,” (a Chinese proverb) embodying strength through flexibility and unity. This concept

mirrors the organic growth seen in software gardens, where a system of interweaved latticework supports the development of intricate and resilient structures.

The lattice also guards against both direct and indirect challenges we face on our journey as software gardeners. These encumbrances can “overheat” our growth potential, leading us to early burnout or burn-down of gardens. The lattice helps us as a carbon sequester, mitigating the excess heat caused by an imbalance in emissions, similar to bamboo garden forests [GL5]. Our individual experiences and personal network consume these difficulties to further grow the lattice through acts of harmonization (such as collaboration or apprenticeship). We care for these experiences together on the lattice because “all flourishing is mutual.” [GL2].

[GL1] Benjamin Gimmel. Round window with lattice. 2005. URL: https://commons.wikimedia.org/wiki/File:Rundes_Fenster_mit_Gitter.jpg (visited on 2024-05-17).

[GL2](1,2) Robin Wall Kimmerer. *Braiding sweetgrass: indigenous wisdom, scientific knowledge and the teachings of plants*. Milkweed Editions, Minneapolis, Minn, first paperback edition edition, 2013. ISBN 978-1-57131-356-0.

[GL3] J. Fuegi and J. Francis. Lovelace & babbage and the creation of the 1843 'notes'. *IEEE Annals of the History of Computing*, 25(4):16–26, October 2003. URL: https://ieeexplore.ieee.org/document/1253887/ (visited on 2024-05-10), doi:10.1109/MAHC.2003.1253887.

[GL4] Frank Herbert and Brian Herbert. *Dune*. The Dune chronicles. Ace, New York, ace premium edition edition, 2010. ISBN 978-0-441-17271-9.

[GL5] Arun Jyoti Nath, Rattan Lal, and Ashesh Kumar Das. Managing woody bamboos for carbon farming and carbon trading. *Global Ecology and Conservation*, 3:654–663, January 2015. URL: https://linkinghub.elsevier.com/retrieve/pii/S2351989415000281 (visited on 2024-05-12), doi:10.1016/j.gecco.2015.03.002.

Understanding

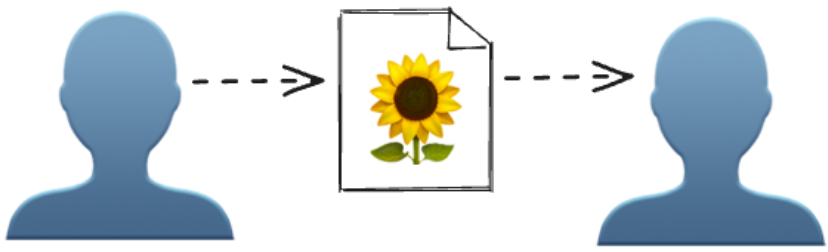


Fig. 5 We transfer understanding of software projects in the form of documentation artifacts.

Shared understanding forms the heartbeat of any successful software project. Peter Naur, in his work “Programming as Theory Building” (1985), emphasized that programming is fundamentally about building a shared theory of the project among all contributors [GLU1]. Understanding encompasses not only the technical aspects of software, but also the collaborative and ethical dimensions that ensure a software project’s growth and sustainability. Just as a gardener must understand the needs of each plant to cultivate a thriving garden, contributors to a project must grasp its goals, structure, and community standards to foster a productive and harmonious environment. Documentation, of which there are many different forms that we describe below, cultivates a shared understanding of a software project.

Common files for shared understanding

In software development, certain files are expected to ensure project scope, clarity, collaboration, and legal compliance. These files serve as the foundational elements that guide contributors and users, much like a well-tended garden benefits from clear paths and signs. When present, these files are also correlated with increased rates of project success [GLU2].

Historically, developers had written the following files in plain-text without formatting. More recently, developers prefer markdown to format the documents with rich styles and media. GitHub, GitLab, and other software source control hosting platforms often automatically render markdown materials in HTML.

README

The `README` file is the cornerstone of any repository, providing essential information about the project. Its presence signifies a well-documented and accessible project, inviting contributors and users alike to understand and engage with the work. A well-structured `README` will reference other pieces of important information that exist elsewhere, such as dependencies and other files we describe below. A repository thrives when its purpose and usage are communicated through a `README` file, much like a garden benefits from a clear plan and understanding of its layout.

> "A good rule of thumb is to assume that the information contained within the README will be the only documentation your users read. For this reason, your README should include how to install and configure your software, where to find its full documentation, under what license it's released, how to test it to ensure functionality, and acknowledgments." [GLU3]

For further reading on `README` file practices, see the following resources:

- The Turing Way: Landing Page - README File
- GitHub: About `README`s
- Makeareadme.com

CONTRIBUTING

A `CONTRIBUTING` file outlines guidelines for how to add to the project. Its presence fosters a welcoming environment for new contributors, ensuring that they have the necessary information to participate effectively. In the same way that a garden flourishes when there's good understanding of how to provide care to the garden and gardeners, a project grows stronger when contributors are guided by clear and inclusive instructions. The resulting nurturing environment fosters growth, resilience, and a sense of community, ensuring the project remains healthy and vibrant.

The `CONTRIBUTING` file should outline all development procedures that are specific to the project. For example, the file might contain testing guidelines, linting/formatting expectations, release cadence, and more. Consider writing this file from the perspective of both an outsider and a beginner: an outsider who might enhance your project by adding a new task or completing an existing task, and a beginner who seeks to understand your project from the ground up [GLU4].

For further reading on `CONTRIBUTING` file practices, see the following resources:

- GitHub: Setting guidelines for repository contributors

- Mozilla: Wrangling Web Contributions: How to Build a CONTRIBUTING.md

CODE_OF_CONDUCT

The `CODE_OF_CONDUCT` (CoC) file sets the standards for behavior within the project community. Its presence signals a commitment to maintaining a respectful and inclusive environment for all participants. It also may provide an outline for acceptable participation when it comes to conflicts of interest. A CoC is a foundational document that defines community values, guides governance and moderation, and signals inclusivity within the project, particularly for vulnerable contributors. [GLU5] A project community benefits from a shared understanding of respectful and supportive interactions, ensuring that all members can contribute positively, much like a garden requires a harmonious ecosystem to thrive.

For further reading and examples of `CODE_OF_CONDUCT` files, see the following:

- Contributor Covenant Code of Conduct
- Example: Python Software Foundation Code of Conduct
- Example: Rust Code of Conduct

LICENSE

A `LICENSE` file specifies the terms under which the project's code can be used, modified, and shared.

> "When you make a creative work (which includes code), the work is under exclusive copyright by default. Unless you include a license that specifies otherwise, nobody else can copy, distribute, or modify your work without being at risk of take-downs, shake-downs, or litigation. Once the work has other contributors (each a copyright holder), "nobody" starts including you." [GLU6]. The presence of a `LICENSE` file is crucial for legal clarity, encourages the responsible use or distribution of the project, and is considered an indicator of project maturity[GLU7].

Just as a garden's health depends on understanding natural laws and respecting boundaries, a project's sustainability hinges on clear licensing that safeguards and empowers both creators and users, cultivating a culture of trust and collaboration.

For further reading and examples of `LICENSE` files, see the following:

- OSF: Licensing
- GitHub: Licensing a repository
- Choosealicense.com
- SPDX License List

Project documentation

Common documentation files like `README`'s, `CONTRIBUTING` guides, and `LICENSE` files are only a start towards more specific project information. Comprehensive project documentation is akin to a detailed gardener's notebook for a well-maintained project, illustrating how the project may be used and the guiding philosophy. This includes in-depth explanations of the project's architecture, practical usage examples, application programming interface (API) references, and development workflows. Oftentimes this type of documentation is

provided through a “documentation website” or “docsite” to facilitate a user experience that includes a search bar, multimedia, and other HTML-enabled features.

Such documentation should strive to ensure that both novice and seasoned contributors can grasp the project’s complexities and contribute effectively by delivering valuable information. Writing valuable content entails conveying information that the code alone cannot communicate to the user [GLU8]. In addition to increased value from understanding, software tends to be more extensively utilized when developers offer more comprehensive documentation [GLU9]. Just as a thriving garden benefits from meticulous care instructions and shared horticultural knowledge, a project flourishes when its documentation offers a clear and thorough guide to its inner workings, nurturing a collaborative and informed community.

Project documentation often exists within a dedicated `docs` directory where the materials may be created and versioned distinctly from other code. Oftentimes this material will leverage a specific documentation tooling technology in alignment with the programming language(s) being used (for example, Python projects often leverage Sphinx). These tools often increase the utility of the output by styling the material with pre-built HTML themes, automating API documentation generation, and an ecosystem of plugins to help extend the capabilities of your documentation without writing new code.

For further reading and examples of deep Project documentation, see the following:

- Berkeley Library: How to Write Good Documentation
- Write the Docs: Software documentation guide
- Overcoming Open Source Project Entry Barriers with a Portal for Newcomers

[GLU1] Peter Naur. Programming as theory building. *Microprocessing and Microprogramming*, 15(5):253–261, May 1985. URL: https://linkinghub.elsevier.com/retrieve/pii/0165607485900328 (visited on 2025-01-02), doi:10.1016/0165-6074(85)90032-8.

[GLU2] Jailton Coelho and Marco Tulio Valente. Why modern open source projects fail. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*, 186–196. Paderborn Germany, August 2017. ACM. URL: https://dl.acm.org/doi/10.1145/3106237.3106246 (visited on 2024-12-31), doi:10.1145/3106237.3106246.

[GLU3] Benjamin D. Lee. Ten simple rules for documenting scientific software. *PLOS Computational Biology*, 14(12):e1006561, December 2018. URL: https://dx.plos.org/10.1371/journal.pcbi.1006561 (visited on 2024-12-31), doi:10.1371/journal.pcbi.1006561.

[GLU4] Christoph Treude, Marco A. Gerosa, and Igor Steinmacher. Towards the First Code Contribution: Processes and Information Needs. 2024. Version Number: 1. URL: https://arxiv.org/abs/2404.18677 (visited on 2024-12-31), doi:10.48550/ARXIV.2404.18677.

[GLU5] Hana Frluckaj and James Howison. Codes of Conduct in Open Source. In Daniela Damian, Kelly Blincoe, Denae Ford, Alexander Serebrenik, and Zainab Masood, editors, *Equity, Diversity, and Inclusion in Software Engineering*, pages 295–308. Apress, Berkeley, CA, 2024. URL: https://link.springer.com/10.1007/978-1-4842-9651-6_17 (visited on 2025-01-02), doi:10.1007/978-1-4842-9651-6_17.

[GLU6] Inc. GitHub. No license. Accessed: 2025-01-02. URL: https://choosealicense.com/no-permission/.

[GLU7] Deekshitha, Rena Bakhshi, Jason Maassen, Carlos Martinez Ortiz, Rob van Nieuwpoort, and Slinger Jansen. RSMM: A Framework to Assess Maturity of Research Software Project. June 2024. arXiv:2406.01788 [cs]. URL: http://arxiv.org/abs/2406.01788 (visited on 2024-10-02).

[GLU8] **missing author in henney_97_2010**

[GLU9] Awan Afiaz, Andrey Ivanov, John Chamberlin, David Hanauer, Candace Savonen, Mary J. Goldman, Martin Morgan, Michael Reich, Alexander Getka, Aaron Holmes, Sarthak Pati, Dan Knight, Paul C. Boutros, Spyridon Bakas, J. Gregory Caporaso, Guilherme Del Fiol, Harry Hochheiser, Brian Haas,

Patrick D. Schloss, James A. Eddy, Jake Albrecht, Andrey Fedorov, Levi Waldron, Ava M. Hoffman, Richard L. Bradshaw, Jeffrey T. Leek, and Carrie Wright. Evaluation of software impact designed for biomedical research: Are we measuring what's meaningful? June 2023. arXiv:2306.03255 [cs, q-bio]. URL: http://arxiv.org/abs/2306.03255 (visited on 2024-10-02).

Software Forest

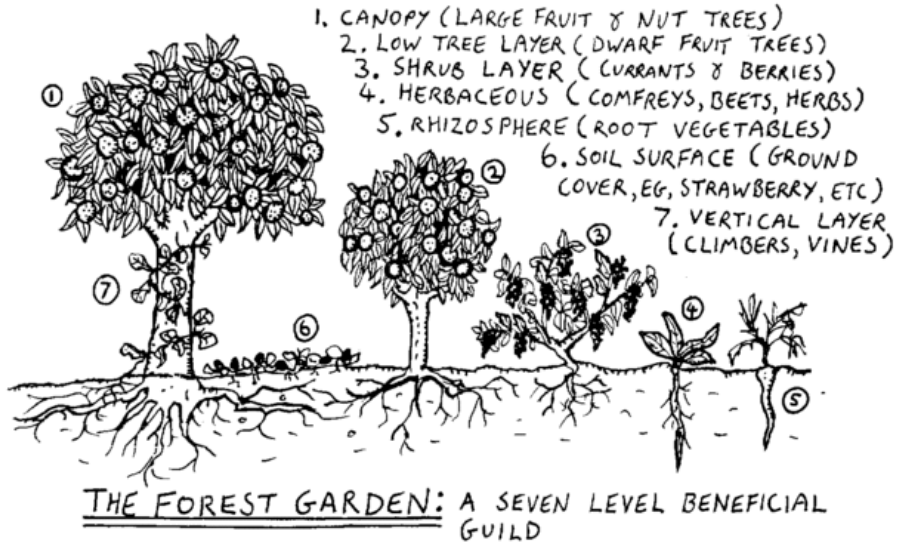


Fig. 6 The software forest is multi-layered like a forest garden. (Image source: SF1).

The software forest is a multidimensional garden, enriched by connections and minimal, iterative changes. Both the practice and artifacts of software development can seem daunting, like an unfamiliar forest full of mystery and surprises. However, through software gardening, we see these forests as opportunities for balanced cultivation.

> "Called Three-Dimensional Forestry or Forest Farming, … the three 'dimensions' of [Toyohiko Kagawa's] 3-D system were the trees as conservers of the soil and suppliers of food and the livestock which benefited from them." SF2, SF3

Forest gardening offers innovations from an ecological cultivation perspective by caring for trees, understory vegetation, soil, and other environmental factors. Forest gardens take time to create because their slow-growing nature requires patience and careful, long-term nurturing. Software forests are also created gradually, "… starting with the existing, living, breathing system, and working outward, a step at a time, in such a way as to not undermine the system's viability." SF4 Nourishing the system in this way helps us practice *primum non nocere* ("first, do no harm") to the garden and components within, conserving energy for necessary adaptations.

Dense networks of relationships surround software forest gardens in the form of software environments. Some of these environments are portable, similar to Wardian cases that provide loosely coupled habitats for software organisms (for example, through environment managers). Others are exhibited through abstract relationships as *nurse logs*, *Hügelkultur*, or mycorrhizal-like associations which redistribute energy among the entirety of a software region.

As software gardeners we must consider how energy is transferred to various portions of the forest. Adventitious code, which “volunteers” by accident of growth, may create scenarios of energy waste in the form of cognitive or performance misalignment. All waste in the forest is food [SF5], therefore we must imagine and craft new homes for wasted energy through appropriate means. This chaos of this soft-scaping (curating the software garden towards a goal) is in constant flux, tugging on our capacities as student and teacher of the garden to encourage growth and avoid harm.

“What gardens demand of us is lifelong learning.” [SF6] We discover and are discovered within the software garden. As software forest gardens grow, they demonstrate emergent properties which are well suited for practices such as evolutionary architecture, object-orientated design, and coupling analysis. These assist software gardeners as a shed of multi-dimensional concepts that must be employed to truly achieve effective multi-layered software garden maintenance.

[SF1] Graham Burnett. Forest garden diagram to replace the one that currently exists on the article. 2006. URL: https://commons.wikimedia.org/wiki/File:Forgard2-003.gif (visited on 2024-05-17).

[SF2] Robert A. de J. Hart. *Forest gardening: cultivating an edible landscape*. Chelsea Green Pub. Co, White River Junction, Vt, 1996. ISBN 978-0-930031-84-8.

[SF3] Kagawa, T. and S. Fujisaki. Rittai nogyo no riron to jissen (theory and practice of three-dimensional structural farming). *Tokyo, Japan: Nihonhyoronsya*, 1935.

[SF4] Brian Foote and Joseph Yoder. Big ball of mud. *Pattern languages of program design*, 4:654–692, 1997.

[SF5] William McDonough. Waste equals food: our future and the making of things. *Awakening–The Upside of Y2k, The Printed Word, Spokane*, 1998.

[SF6] Viviane Stappmanns, Mateo Kries, Vitra Design Museum, and Wüstenrot Stiftung, editors. *Garden futures: designing with nature*. Vitra Design Museum, Weil am Rhein, 2023. ISBN 978-3-945852-53-8.

Verdant Sundial



Fig. 7 The verdant sundial casts shadows of past and future intention observed in the present. (Image source: VS1).

> “The present is the ever moving shadow that divides yesterday from tomorrow. In that lies hope.”
> - Frank Lloyd Wright

The software garden is intertwined in a story of time which is exhibited like a shadow on a moss-covered sundial. Time is observed indirectly through a kind of software archeology in past-tense material artifacts (files) and present-tense operation (procedures). These artifacts act as shadows of our prior creative intentions where procedures become the living present materialized promise of those intentions. “This brings an uncanny sense of living in a state where the future is a dimension of our present reality.” [VS2] The future is elusive but present in software artifacts, similar to rings of a tree that depict the promise of continued stability or hope of change.

These software tree rings are vessels of temporal data which offer a unique insight into growth patterns. We can study these patterns like dendronchronology: “Peel away the hard, rough bark and there is a living document, history recorded in rings of wood cells.” [VS3] This peeling helps us understand the chronological nature of software and how it evolves. It also shares patterns of temporal pulsing - not all times of the software garden are the same.

These software pulses give visibility into distinct growing or receding patterns. Some code exhibits annual behavior, requiring continual replanting efforts each season. Others are perennials, undergoing seasonal change perpetuated by periods of vibrancy and dormancy. Seasons of software promote differing patterns among these. Some systems may prepare for winter (for instance, during resource scarcity, or periods of high performance stress) through adaptable conservation only to fervently bloom during their spring.

Seasonal chronology in software benefits also from an awareness of kairos, the opportune timing of our actions amidst time. As we gain experience in software gardens we begin to anticipate seasonal change which helps us project preparatory change. Each software gardening moment amidst this cascade of season includes a series of options of varying kairological fitness. Assessing these moments of their fit within patterns helps us gain wisdom as we contemplate time in software gardens; like annual flowers, how do we anticipate and accept the limited lifecycle of some software? Like mighty trees, how do we build software to last 100 years or more?

[VS1] Elfward. Garden sundial in an epping garden. 2015. URL: https://commons.wikimedia.org/wiki/File:Sundial_2916_HDR.jpg (visited on 2024-05-18).

[VS2] Timothy Morton. *Hyperobjects: Philosophy and Ecology after the End of the World*. University of Minnesota Press, Minneapolis, 2013. ISBN 9780816689231.

[VS3] Carolyn Gramling. ‘tree story’ explores what tree rings can tell us about the past. *Science News*, June 2020. Accessed: 2024-05-18. URL: https://www.sciencenews.org/article/tree-story-book-explores-what-tree-rings-can-tell-us-about-past.

Seed Bank

The seed bank is a section of the Software Garden Almanack associated with applied demonstrations of concepts and technologies associated with other areas of content.

Software Gardening Almanack Example

This notebook demonstrates installing and using the Software Gardening Almanack Python package. The Almanack is an open-source handbook of applied guidance and tools for sustainable software development and maintenance.

```
# install the almanack from pypi
import json

import pandas as pd

import almanack

!pip install almanack
```

```
Requirement already satisfied: almanack in /opt/hostedtoolcache/Python/3.11.18/x64/lib/python3.11/site-pa
Requirement already satisfied: charset-normalizer<4,>=3.4 in /opt/hostedtoolcache/Python/3.11.18/x64/lib/
Requirement already satisfied: currencyconverter<0.19,>=0.18.15 in /opt/hostedtoolcache/Python/3.11.18/x6
Requirement already satisfied: defusedxml<0.8,>=0.7.1 in /opt/hostedtoolcache/Python/3.11.18/x64/lib/pyth
Requirement already satisfied: fire<0.8,>=0.6 in /opt/hostedtoolcache/Python/3.11.18/x64/lib/python3.11/s
Requirement already satisfied: pandas>=2.2.2 in /opt/hostedtoolcache/Python/3.11.18/x64/lib/python3.11/si
Requirement already satisfied: pyarrow>=16 in /opt/hostedtoolcache/Python/3.11.18/x64/lib/python3.11/site
Requirement already satisfied: pygit2>=1.15.1 in /opt/hostedtoolcache/Python/3.11.18/x64/lib/python3.11/s
Requirement already satisfied: pyyaml<7,>=6.0.1 in /opt/hostedtoolcache/Python/3.11.18/x64/lib/python3.11
Requirement already satisfied: requests<3,>=2.32.3 in /opt/hostedtoolcache/Python/3.11.18/x64/lib/python3
Requirement already satisfied: tabulate<0.11,>=0.9 in /opt/hostedtoolcache/Python/3.11.18/x64/lib/python3
Requirement already satisfied: termcolor in /opt/hostedtoolcache/Python/3.11.18/x64/lib/python3.11/site-p
Requirement already satisfied: idna<4,>=2.5 in /opt/hostedtoolcache/Python/3.11.18/x64/lib/python3.11/sit
Requirement already satisfied: urllib3<3,>=1.26 in /opt/hostedtoolcache/Python/3.11.18/x64/lib/python3.11
Requirement already satisfied: certifi>=2023.5.7 in /opt/hostedtoolcache/Python/3.11.18/x64/lib/python3.1
Requirement already satisfied: numpy>=2.0.2 in /opt/hostedtoolcache/Python/3.11.18/x64/lib/python3.11/sit
Requirement already satisfied: python-dateutil>=2.9.0 in /opt/hostedtoolcache/Python/3.11.18/x64/lib/pyth
Requirement already satisfied: cffi>=2.0 in /opt/hostedtoolcache/Python/3.11.18/x64/lib/python3.11/site-p
Requirement already satisfied: pycparser in /opt/hostedtoolcache/Python/3.11.18/x64/lib/python3.11/site-p
Requirement already satisfied: tzdata in /opt/hostedtoolcache/Python/3.11.18/x64/lib/python3.11/site-pacl

[notice] A new release of pip is available: 26.2.2 -> 26.3
[notice] To update, run: pip install --upgrade pip
```

```
# clone the almanack repo for example usage below
# we will apply the almanack tool to the almanack repo
!git clone https://github.com/software-gardening/almanack
```

```
fatal: destination path 'almanack' already exists and is not an empty directory.
```

```
%%time


# gather the almanack table using the almanack repo as a reference
almanack_table = almanack.metrics.data.get_table("almanack")

# show the almanack table as a Pandas DataFrame
pd.DataFrame(almanack_table)
```

```
Transient HTTP 503 for https://api.openalex.org/works (attempt 1/3). Retrying in 2 seconds...
```

```
Transient HTTP 503 for https://api.openalex.org/works (attempt 2/3). Retrying in 4 seconds...
```

```
CPU times: user 5.5 s, sys: 395 ms, total: 5.89 s
Wall time: 20.1 s
```

	name	id	result-type	sustainability_correlation	description	correction_guidance	fix_why	fix_
0	repo-path	SGA-META-0001	str	0	Repository path (local directory).	NaN	NaN	NaN
1	repo-commits	SGA-META-0002	int	0	Total number of commits for the repository.	NaN	NaN	NaN
2	repo-file-count	SGA-META-0003	int	0	Total number of files tracked within the repos...	NaN	NaN	NaN
3	repo-commit-time-range	SGA-META-0004	tuple	0	Starting commit and most recent commit for the...	NaN	NaN	NaN
4	repo-days-of-development	SGA-META-0005	int	0	Integer representing the number of days of dev...	NaN	NaN	NaN
...	...	...	...	...	...	...	...	...
95	repo-code-coverage-executed-lines	SGA-SF-0009	int	0	Total lines covered code within repository giv...	NaN	NaN	NaN
96	repo-agg-info-entropy	SGA-VS-0001	float	0	Aggregated information entropy for all files w...	NaN	NaN	NaN
97	repo-file-info-entropy	SGA-VS-0002	dict	0	File-level information entropy for all files w...	NaN	NaN	NaN
98	repo-agg-history-complexity-decay	SGA-VS-0003	float	0	Aggregated history complexity with decay (HCM_...	NaN	NaN	NaN
99	repo-file-history-complexity-decay	SGA-VS-0004	dict	0	File-level history complexity with decay (HCM_...	NaN	NaN	NaN

100 rows × 9 columns

```
# print the json with indentation
print(json.dumps(almanack_table, indent=4))
```

```
[
    {
        "name": "repo-path",
        "id": "SGA-META-0001",
        "result-type": "str",
        "sustainability_correlation": 0,
        "description": "Repository path (local directory).",
        "correction_guidance": null,
        "fix_why": null,
        "fix_how": null,
        "result": "/home/runner/work/almanack/almanack/src/_pdfbuild/seed-bank/almanack-example/almanack"
    },
    {
        "name": "repo-commits",
        "id": "SGA-META-0002",
        "result-type": "int",
        "sustainability_correlation": 0,
        "description": "Total number of commits for the repository.",
        "correction_guidance": null,
        "fix_why": null,
        "fix_how": null,
        "result": 351
    },
    {
        "name": "repo-file-count",
        "id": "SGA-META-0003",
        "result-type": "int",
        "sustainability_correlation": 0,
        "description": "Total number of files tracked within the repository.",
        "correction_guidance": null,
        "fix_why": null,
        "fix_how": null,
        "result": 192
    },
    {
        "name": "repo-commit-time-range",
        "id": "SGA-META-0004",
        "result-type": "tuple",
        "sustainability_correlation": 0,
        "description": "Starting commit and most recent commit for the repository.",
        "correction_guidance": null,
        "fix_why": null,
        "fix_how": null,
        "result": [
            "2024-03-05",
            "2026-11-05"
        ]
    },
    {
        "name": "repo-days-of-development",
        "id": "SGA-META-0005",
        "result-type": "int",
        "sustainability_correlation": 0,
        "description": "Integer representing the number of days of development between most recent commit
        "correction_guidance": null,
        "fix_why": null,
        "fix_how": null,
        "result": 976
    },
    {
        "name": "repo-commits-per-day",
        "id": "SGA-META-0006",
        "result-type": "float",
        "sustainability_correlation": 0,
        "description": "Floating point number which represents the number of commits per day (using days
        "correction_guidance": null,
        "fix_why": null,
        "fix_how": null,
        "result": 0.35963114754098363
    },
    {
        "name": "almanack-table-datetime",
        "id": "SGA-META-0007",
```

```
    "result-type": "str",
    "sustainability_correlation": 0,
    "description": "String representing the date when this table was generated in the format of '%Y-%
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": "2026-11-05T15:26:31.057688Z"
},
{
    "name": "almanack-version",
    "id": "SGA-META-0008",
    "result-type": "str",
    "sustainability_correlation": 0,
    "description": "String representing the version of the almanack which was used to generate this t
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": "0.1.dev1+g39a329bd0"
},
{
    "name": "repo-software-description",
    "id": "SGA-META-0018",
    "result-type": "str",
    "sustainability_correlation": 0,
    "description": "Short free-text description of the software project, assembled from repository ho
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": "An open-source handbook of applied guidance and tools for sustainable software develop
},
{
    "name": "repo-docs-homepage-url",
    "id": "SGA-META-0019",
    "result-type": "str",
    "sustainability_correlation": 0,
    "description": "URL of the primary documentation or project homepage, when available. Derived fro
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": "https://software-gardening.github.io/almanack"
},
{
    "name": "repo-source-code-url",
    "id": "SGA-META-0020",
    "result-type": "str",
    "sustainability_correlation": 0,
    "description": "Canonical URL pointing to the primary source code repository for the software pro
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": "https://github.com/software-gardening/almanack"
},
{
    "name": "repo-issue-tracker-url",
    "id": "SGA-META-0021",
    "result-type": "str",
    "sustainability_correlation": 0,
    "description": "URL for the primary issue tracker associated with the software project. This may
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": "https://github.com/software-gardening/almanack/issues"
},
{
    "name": "repo-primary-language",
    "id": "SGA-META-0009",
    "result-type": "str",
    "sustainability_correlation": 0,
    "description": "Detected primary programming language of the repository.",
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": "Jupyter Notebook"
},
```

```
{
    "name": "repo-primary-license",
    "id": "SGA-META-0010",
    "result-type": "str",
    "sustainability_correlation": 0,
    "description": "Detected primary license of the repository.",
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": "bsd-3-clause"
},
{
    "name": "repo-doi",
    "id": "SGA-META-0011",
    "result-type": "str",
    "sustainability_correlation": 0,
    "description": "Repository DOI value detected from CITATION.cff file.",
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": "10.5281/zenodo.14765834"
},
{
    "name": "repo-doi-publication-date",
    "id": "SGA-META-0012",
    "result-type": "str",
    "sustainability_correlation": 0,
    "description": "Repository DOI publication date detected from CITATION.cff file and OpenAlex.org.
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": "2025-01-29T00:00:00.000000Z"
},
{
    "name": "repo-doi-fwci",
    "id": "SGA-META-0013",
    "result-type": "float",
    "sustainability_correlation": 0,
    "description": "Field-Weighted Citation Impact (FWCI) from OpenAlex for the work referenced by th
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": null
},
{
    "name": "repo-doi-is-not-retracted",
    "id": "SGA-META-0014",
    "result-type": "bool",
    "sustainability_correlation": 0,
    "description": "If the work referenced by the repository DOI has been retracted as reported by Op
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": true
},
{
    "name": "repo-doi-grants-count",
    "id": "SGA-META-0015",
    "result-type": "int",
    "sustainability_correlation": 0,
    "description": "Count of grant records attached to the DOI's work (from OpenAlex via Crossref).",
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": 0
},
{
    "name": "repo-doi-grants",
    "id": "SGA-META-0016",
    "result-type": "list",
    "sustainability_correlation": 0,
    "description": "List of grant records for the DOI's work (from OpenAlex via Crossref).",
    "correction_guidance": null,
    "fix_why": null,
```

```
    "fix_how": null,
    "result": []
},
{
    "name": "repo-almanack-score",
    "id": "SGA-META-0017",
    "result-type": "dict",
    "sustainability_correlation": 0,
    "description": "Dictionary of length three, including the following: 1) number of Almanack boolea
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": {
        "almanack-score-numerator": 10,
        "almanack-score-denominator": 11,
        "almanack-score": 0.9090909090909091
    }
},
{
    "name": "repo-includes-readme",
    "id": "SGA-GL-0001",
    "result-type": "bool",
    "sustainability_correlation": 1,
    "description": "Boolean value indicating the presence of a README file in the repository.",
    "correction_guidance": "Consider adding a README file to the repository.",
    "fix_why": "A README file is essential for providing an overview of the project, its purpose, and
    "fix_how": "Create a README file in the root directory of your repository. Include sections such
    "result": true
},
{
    "name": "repo-includes-contributing",
    "id": "SGA-GL-0002",
    "result-type": "bool",
    "sustainability_correlation": 1,
    "description": "Boolean value indicating the presence of a CONTRIBUTING file in the repository.",
    "correction_guidance": "Consider adding a CONTRIBUTING file to the repository.",
    "fix_why": "A CONTRIBUTING file is important for guiding potential contributors on how to contrib
    "fix_how": "Create a CONTRIBUTING file in your repository. Include guidelines for contributing, s
    "result": true
},
{
    "name": "repo-includes-code-of-conduct",
    "id": "SGA-GL-0003",
    "result-type": "bool",
    "sustainability_correlation": 1,
    "description": "Boolean value indicating the presence of a CODE_OF_CONDUCT file in the repository
    "correction_guidance": "Consider adding a CODE_OF_CONDUCT file to the repository.",
    "fix_why": "A CODE_OF_CONDUCT file is essential for establishing a respectful and inclusive commu
    "fix_how": "Create a CODE_OF_CONDUCT file in the root directory of your repository. Include guide
    "result": true
},
{
    "name": "repo-includes-license",
    "id": "SGA-GL-0004",
    "result-type": "bool",
    "sustainability_correlation": 1,
    "description": "Boolean value indicating the presence of a LICENSE file in the repository.",
    "correction_guidance": "Consider adding a LICENSE file to the repository.",
    "fix_why": "A LICENSE file is crucial for defining the terms under which the code can be used, mo
    "fix_how": "Create a LICENSE file in the root directory of your repository. Include the text of t
    "result": true
},
{
    "name": "repo-is-citable",
    "id": "SGA-GL-0005",
    "result-type": "bool",
    "sustainability_correlation": 1,
    "description": "Boolean value indicating the presence of a CITATION file or some other means of i
    "correction_guidance": "Consider adding a CITATION file to the repository (for example, citation.
    "fix_why": "A CITATION file is important for providing a standardized way to cite the work, makin
    "fix_how": "Create a CITATION file in the root directory of your repository. Include information
    "result": true
},
{
```

```
    "name": "repo-default-branch-not-master",
    "id": "SGA-GL-0006",
    "result-type": "bool",
    "sustainability_correlation": 1,
    "description": "Boolean value indicating that the repo uses a default branch name besides 'master
    "correction_guidance": "Consider using a default source control branch name other than 'master'.'
    "fix_why": "Using a default branch name other than 'master' is more collaborative and avoids poss
    "fix_how": "Change the default branch name in your source control platform (for example, GitHub,
    "result": true
},
{
    "name": "repo-includes-common-docs",
    "id": "SGA-GL-0007",
    "result-type": "bool",
    "sustainability_correlation": 1,
    "description": "Boolean value indicating whether the repo includes common documentation directory
    "correction_guidance": "Consider including project documentation through the 'docs/' directory.",
    "fix_why": "Including a 'docs/' directory with common documentation files helps in maintaining a
    "fix_how": "Create a 'docs/' directory in the root of your repository. Inside this directory, inc
    "result": true
},
{
    "name": "repo-unique-contributors",
    "id": "SGA-GL-0008",
    "result-type": "int",
    "sustainability_correlation": 0,
    "description": "Count of unique contributors since the beginning of the repository.",
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": 11
},
{
    "name": "repo-unique-contributors-past-year",
    "id": "SGA-GL-0009",
    "result-type": "int",
    "sustainability_correlation": 0,
    "description": "Count of unique contributors within the last year from now (where now is a refere
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": 9
},
{
    "name": "repo-unique-contributors-past-182-days",
    "id": "SGA-GL-0010",
    "result-type": "int",
    "sustainability_correlation": 0,
    "description": "Count of unique contributors within the last 182 days from now (where now is a re
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": 6
},
{
    "name": "repo-tags-count",
    "id": "SGA-GL-0011",
    "result-type": "int",
    "sustainability_correlation": 0,
    "description": "Count of the number of tags within the repository.",
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": 23
},
{
    "name": "repo-tags-count-past-year",
    "id": "SGA-GL-0012",
    "result-type": "int",
    "sustainability_correlation": 0,
    "description": "Count of the number of tags within the repository within the last year from now
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
```

```
    "result": 8
},
{
    "name": "repo-tags-count-past-182-days",
    "id": "SGA-GL-0013",
    "result-type": "int",
    "sustainability_correlation": 0,
    "description": "Count of the number of tags within the repository within the last 182 days from r
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": 1
},
{
    "name": "repo-stargazers-count",
    "id": "SGA-GL-0014",
    "result-type": "int",
    "sustainability_correlation": 0,
    "description": "Count of the number of stargazers on repository remote hosting platform.",
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": 26
},
{
    "name": "repo-uses-issues",
    "id": "SGA-GL-0015",
    "result-type": "bool",
    "sustainability_correlation": 1,
    "description": "Whether the repository uses issues (for example, as a bug and/or feature tracking
    "correction_guidance": "Consider leveraging issues for tracking bugs and/or features within your
    "fix_why": "Using issues for tracking bugs and features helps in maintaining a clear record of pr
    "fix_how": "Enable issues on your repository hosting platform (for example, GitHub, GitLab, Bitbu
    "result": true
},
{
    "name": "repo-issues-open-count",
    "id": "SGA-GL-0016",
    "result-type": "int",
    "sustainability_correlation": 0,
    "description": "Count of open issues for repository.",
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": 72
},
{
    "name": "repo-pull-requests-enabled",
    "id": "SGA-GL-0017",
    "result-type": "bool",
    "sustainability_correlation": 1,
    "description": "Whether the repository enables pull requests.",
    "correction_guidance": "Consider enabling pull requests for the repository through your repositor
    "fix_why": null,
    "fix_how": null,
    "result": true
},
{
    "name": "repo-forks-count",
    "id": "SGA-GL-0018",
    "result-type": "int",
    "sustainability_correlation": 0,
    "description": "Count of forks of the repository.",
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": 6
},
{
    "name": "repo-subscribers-count",
    "id": "SGA-GL-0019",
    "result-type": "int",
    "sustainability_correlation": 0,
    "description": "Count of subscribers (or watchers) of the repository.",
```

```
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": 1
},
{
    "name": "repo-packages-ecosystems",
    "id": "SGA-GL-0020",
    "result-type": "list",
    "sustainability_correlation": 0,
    "description": "List of package platforms or services where the repository was detected (leveragi
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": [
        "pypi"
    ]
},
{
    "name": "repo-packages-ecosystems-count",
    "id": "SGA-GL-0021",
    "result-type": "int",
    "sustainability_correlation": 0,
    "description": "Count of package platforms or services where the repository was detected (leverag
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": 1
},
{
    "name": "repo-packages-versions-count",
    "id": "SGA-GL-0022",
    "result-type": "int",
    "sustainability_correlation": 0,
    "description": "Count of package versions on package hosts or services where the repository was d
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": 24
},
{
    "name": "repo-social-media-platforms",
    "id": "SGA-GL-0023",
    "result-type": "list",
    "sustainability_correlation": 0,
    "description": "Social media platforms detected within the readme.",
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": []
},
{
    "name": "repo-social-media-platforms-count",
    "id": "SGA-GL-0024",
    "result-type": "int",
    "sustainability_correlation": 0,
    "description": "Count of social media platforms detected within the readme.",
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": 0
},
{
    "name": "repo-doi-valid-format",
    "id": "SGA-GL-0025",
    "result-type": "bool",
    "sustainability_correlation": 1,
    "description": "Whether the DOI found in the CITATION.cff file is of a valid format.",
    "correction_guidance": "DOI within the CITATION.cff file is not of a valid format or missing.",
    "fix_why": "A valid DOI format is essential for ensuring that the DOI can be resolved and used to
    "fix_how": "Ensure that the DOI in the CITATION.cff file follows the standard DOI format, which t
    "result": true
},
{
```

```
    "name": "repo-doi-https-resolvable",
    "id": "SGA-GL-0026",
    "result-type": "bool",
    "sustainability_correlation": 0,
    "description": "Whether the DOI found in the CITATION.cff file is HTTPS resolvable.",
    "correction_guidance": "DOI within the CITATION.cff file is not HTTPS resolvable or missing.",
    "fix_why": "An HTTPS resolvable DOI is crucial for ensuring that the DOI can be accessed securely
    "fix_how": "Verify that the DOI in the CITATION.cff file can be resolved using an HTTPS URL. The
    "result": true
},
{
    "name": "repo-doi-cited-by-count",
    "id": "SGA-GL-0027",
    "result-type": "int",
    "sustainability_correlation": 0,
    "description": "How many other works cite the DOI for the repository found within the CITATION.ct
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": 3
},
{
    "name": "repo-days-between-doi-publication-date-and-latest-commit",
    "id": "SGA-GL-0028",
    "result-type": "int",
    "sustainability_correlation": 0,
    "description": "The number of days between the most recent commit and DOI for the repository's pu
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": 645
},
{
    "name": "repo-check-notebook-dir",
    "id": "SGA-GL-0029",
    "result-type": "bool",
    "sustainability_correlation": 0,
    "description": "Checks whether there is a dedicated `notebooks/` directory.",
    "correction_guidance": "Consider creating a dedicated `notebooks/` directory to organize your Jup
    "fix_why": "Having a `notebooks/` directory is required as it provides a clear and intuitive stru
    "fix_how": "Create a `notebooks/` directory in the root of your project and move all Jupyter note
    "result": false
},
{
    "name": "repo-check-notebook-exec-order",
    "id": "SGA-GL-0030",
    "result-type": "bool",
    "sustainability_correlation": 1,
    "description": "Checks whether the notebooks cells are executed in sequential order.",
    "correction_guidance": "Consider executing notebook cells in sequential order to improve reproduc
    "fix_why": "Ensuring that notebook cells are executed in order improves readability and maintains
    "fix_how": "Open your notebook and use the \"Run All\" or \"Restart & Run All\" option in your no
    "result": false
},
{
    "name": "repo-software-mentions-count",
    "id": "SGA-GL-0031",
    "result-type": "int",
    "sustainability_correlation": 0,
    "description": "Aggregated count of OpenAlex works matching the repository software name. This is
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": null
},
{
    "name": "repo-software-mentions",
    "id": "SGA-GL-0032",
    "result-type": "dict",
    "sustainability_correlation": 0,
    "description": "Minimal OpenAlex reference details for works matching the repository software nam
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
```

```
    "result": {
        "query": "almanack",
        "mentions_count": null,
        "references": null
    }
},
{
    "name": "repo-funding-details",
    "id": "SGA-GL-0033",
    "result-type": "dict",
    "sustainability_correlation": 0,
    "description": "Funding details for the software's DOI-linked work, derived from OpenAlex using t
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": {
        "doi": "10.5281/zenodo.14765834",
        "source_work_id": "https://openalex.org/W6911960640",
        "doi_work_funding_records_count": 0,
        "doi_work_funding_amount_usd_total": 0.0,
        "doi_work_funding_sources_count": 0,
        "doi_work_unique_funders_count": 0,
        "doi_work_unique_funders": [],
        "funding_records": []
    }
},
{
    "name": "repo-funding-details-of-citing-works",
    "id": "SGA-GL-0034",
    "result-type": "dict",
    "sustainability_correlation": 0,
    "description": "Funding details from OpenAlex works that cite the software's DOI-linked work.",
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": {
        "source_work_id": "https://openalex.org/W6911960640",
        "citing_works_count_total": 3,
        "citing_works_count_sampled": 3,
        "citing_works_with_funding_count": 3,
        "citing_works_funding_records_count_sampled": 7,
        "citing_works_funding_amount_usd_total_sampled": 0.0,
        "citing_works_funding_sources_count_sampled": 0,
        "citing_works_unique_funders_count_sampled": 6,
        "citing_works_unique_funders_sampled": [
            "https://openalex.org/F4320306076",
            "https://openalex.org/F4320306084",
            "https://openalex.org/F4320306230",
            "https://openalex.org/F4320306589",
            "https://openalex.org/F4320316728",
            "https://openalex.org/F4320332369"
        ],
        "sample_limit": 25,
        "references": [
            {
                "id": "https://openalex.org/W7143483365",
                "title": "Scalable data harmonization for single-cell image-based profiling with Cyt
                "doi": "https://doi.org/10.1016/j.patter.2026.101514",
                "publication_year": 2026,
                "cited_by_count": 6,
                "funding_records_count": 2,
                "funding_amount_usd_total": 0.0,
                "funding_sources_count": 0,
                "unique_funders_count": 2
            },
            {
                "id": "https://openalex.org/W4415202725",
                "title": "Stellar quality control for single-cell image-based profiling with coSMicQ
                "doi": "https://doi.org/10.1101/2025.10.14.682427",
                "publication_year": 2025,
                "cited_by_count": 3,
                "funding_records_count": 2,
                "funding_amount_usd_total": 0.0,
                "funding_sources_count": 0,
```

```
                "unique_funders_count": 3
            },
            {
                "id": "https://openalex.org/W7103994138",
                "title": "Sustaining Growth of Scientific Software with the Software Gardening Almana
                "doi": "https://doi.org/10.5281/zenodo.17527521",
                "publication_year": 2025,
                "cited_by_count": 0,
                "funding_records_count": 3,
                "funding_amount_usd_total": 0.0,
                "funding_sources_count": 0,
                "unique_funders_count": 3
            }
        ]
    }
},
{
    "name": "repo-funding-count",
    "id": "SGA-GL-0035",
    "result-type": "int",
    "sustainability_correlation": 0,
    "description": "Count of funding records on the OpenAlex work linked to the software project's DO
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": 0
},
{
    "name": "repo-funding-count-of-citing-works",
    "id": "SGA-GL-0036",
    "result-type": "int",
    "sustainability_correlation": 0,
    "description": "Sampled count of funding records across queried OpenAlex works that cite the soft
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": 7
},
{
    "name": "repo-funding-amount-usd",
    "id": "SGA-GL-0037",
    "result-type": "float",
    "sustainability_correlation": 0,
    "description": "Estimated funding amount total in USD for the OpenAlex work linked to the softwar
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": 0.0
},
{
    "name": "repo-funding-amount-usd-of-citing-works",
    "id": "SGA-GL-0038",
    "result-type": "float",
    "sustainability_correlation": 0,
    "description": "Estimated sampled funding amount total in USD across queried OpenAlex citing work
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": 0.0
},
{
    "name": "repo-funding-amount-usd-combined",
    "id": "SGA-GL-0039",
    "result-type": "float",
    "sustainability_correlation": 0,
    "description": "Combined funding amount in USD, computed as: `repo-funding-amount-usd + repo-func
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": 0.0
},
{
    "name": "repo-funder-references-count",
    "id": "SGA-GL-0040",
```

```
    "result-type": "int",
    "sustainability_correlation": 0,
    "description": "Count of funder references discovered on the OpenAlex work linked to the software
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": 0
},
{
    "name": "repo-funder-references-count-of-citing-works",
    "id": "SGA-GL-0041",
    "result-type": "int",
    "sustainability_correlation": 0,
    "description": "Count of sampled funder references discovered across queried OpenAlex citing work
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": 0
},
{
    "name": "repo-funder-references-count-combined",
    "id": "SGA-GL-0042",
    "result-type": "int",
    "sustainability_correlation": 0,
    "description": "Combined funder-reference count, computed as: `repo-funder-references-count + rep
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": 0
},
{
    "name": "repo-unique-funders-count",
    "id": "SGA-GL-0043",
    "result-type": "int",
    "sustainability_correlation": 0,
    "description": "Count of unique funders discovered on the OpenAlex work linked to the software pr
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": 0
},
{
    "name": "repo-unique-funders-count-of-citing-works",
    "id": "SGA-GL-0044",
    "result-type": "int",
    "sustainability_correlation": 0,
    "description": "Count of unique sampled funders discovered across queried OpenAlex citing works.'
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": 6
},
{
    "name": "repo-unique-funders-count-combined",
    "id": "SGA-GL-0045",
    "result-type": "int",
    "sustainability_correlation": 0,
    "description": "Combined unique-funder count across DOI work and sampled citing works (set-union
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": 6
},
{
    "name": "repo-languages-line-counts",
    "id": "SGA-SF-0012",
    "result-type": "dict",
    "sustainability_correlation": 0,
    "description": "Mapping from detected programming language name to an approximate line or byte co
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": {
        "Jupyter Notebook": 1323525,
```

```
        "Python": 335303,
        "TeX": 18315,
        "Typst": 7599,
        "CSS": 483,
        "Jinja": 107
    }
},
{
    "name": "repo-languages-total-lines",
    "id": "SGA-SF-0013",
    "result-type": "int",
    "sustainability_correlation": 0,
    "description": "Aggregate total of language line/byte counts across all detected programming lang
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": 1685332
},
{
    "name": "repo-languages-count",
    "id": "SGA-SF-0014",
    "result-type": "int",
    "sustainability_correlation": 0,
    "description": "Number of distinct programming languages detected for the repository based on hos
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": 6
},
{
    "name": "repo-noncode-extension-line-counts",
    "id": "SGA-SF-0015",
    "result-type": "dict",
    "sustainability_correlation": 0,
    "description": "Mapping from non-programming file extension (for example, .md, .txt, .csv, .png)
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": {
        "<no_ext>": 321,
        ".yml": 1807,
        ".yaml": 199,
        ".ini": 31,
        ".cff": 216,
        ".txt": 61,
        ".in": 61,
        ".xml": 2602,
        ".svg": 1,
        ".json": 530,
        ".toml": 212,
        ".pdf": 14678257,
        ".png": 11469854,
        ".qmd": 125,
        ".odp": 59821889,
        ".css": 22,
        ".j2": 10,
        ".gif": 78106,
        ".jpeg": 206847,
        ".bib": 303,
        ".ipynb": 2923,
        ".parquet": 5576897,
        ".lcov": 999,
        ".lock": 3255
    }
},
{
    "name": "repo-noncode-total-lines",
    "id": "SGA-SF-0016",
    "result-type": "int",
    "sustainability_correlation": 0,
    "description": "Aggregate total of approximate line or size counts across all detected non-progra
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
```

```
    "result": 91845528
},
{
    "name": "repo-noncode-extensions-count",
    "id": "SGA-SF-0017",
    "result-type": "int",
    "sustainability_correlation": 0,
    "description": "Number of distinct non-programming file extensions detected when scanning the rep
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": 24
},
{
    "name": "repo-has-cli",
    "id": "SGA-SF-0018",
    "result-type": "bool",
    "sustainability_correlation": 0,
    "description": "Indicates whether the software exposes at least one command-line interface (CLI)
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": true
},
{
    "name": "repo-cli-entrypoints",
    "id": "SGA-SF-0019",
    "result-type": "list",
    "sustainability_correlation": 0,
    "description": "List of CLI commands used to invoke the software, as inferred from packaging meta
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": [
        "almanack"
    ]
},
{
    "name": "repo-topics",
    "id": "SGA-GL-0046",
    "result-type": "list",
    "sustainability_correlation": 0,
    "description": "List of descriptive topics, labels, or tags associated with the software project
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": [
        "open-source",
        "software-sustainability"
    ]
},
{
    "name": "repo-topics-count",
    "id": "SGA-GL-0047",
    "result-type": "int",
    "sustainability_correlation": 0,
    "description": "Count of descriptive topics, labels, or tags associated with the software project
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": 2
},
{
    "name": "repo-environment-managers",
    "id": "SGA-SF-0021",
    "result-type": "list",
    "sustainability_correlation": 0,
    "description": "List of detected environment management tools configured for the software project
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": null
},
{
```

```
    "name": "repo-has-managed-environment",
    "id": "SGA-SF-0022",
    "result-type": "bool",
    "sustainability_correlation": 0,
    "description": "Indicates whether the software project appears to use at least one explicit envi
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": null
},
{
    "name": "repo-dependency-managers",
    "id": "SGA-SF-0010",
    "result-type": "list",
    "sustainability_correlation": 0,
    "description": "List of detected dependency management tools configured for the software project
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": null
},
{
    "name": "repo-has-declared-dependencies",
    "id": "SGA-SF-0011",
    "result-type": "bool",
    "sustainability_correlation": 0,
    "description": "Indicates whether the software project declares its dependencies via at least one
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": null
},
{
    "name": "repo-declared-python-versions",
    "id": "SGA-SF-0023",
    "result-type": "list",
    "sustainability_correlation": 0,
    "description": "Python version constraints declared by the project in packaging or environment co
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": null
},
{
    "name": "repo-cost-model",
    "id": "SGA-GL-0048",
    "result-type": "str",
    "sustainability_correlation": 0,
    "description": "High-level cost model for the software project (for example, free, open-source wi
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": null
},
{
    "name": "repo-has-edam-owl",
    "id": "SGA-SF-0024",
    "result-type": "bool",
    "sustainability_correlation": 0,
    "description": "Indicates whether the repository contains an EDAM ontology file (for example, eda
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": false
},
{
    "name": "repo-has-biotools-json",
    "id": "SGA-SF-0025",
    "result-type": "bool",
    "sustainability_correlation": 0,
    "description": "Indicates whether the repository includes a bio.tools JSON metadata file (for exa
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
```

```
    "result": false
},
{
    "name": "repo-has-codemeta-json",
    "id": "SGA-SF-0026",
    "result-type": "bool",
    "sustainability_correlation": 0,
    "description": "Indicates whether the repository includes a CodeMeta JSON file (for example, code
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": false
},
{
    "name": "repo-has-ro-crate-metadata-json",
    "id": "SGA-SF-0027",
    "result-type": "bool",
    "sustainability_correlation": 0,
    "description": "Indicates whether the repository includes an RO-Crate metadata file (for example,
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": false
},
{
    "name": "repo-software-metadata-files-count",
    "id": "SGA-SF-0028",
    "result-type": "int",
    "sustainability_correlation": 0,
    "description": "Count of distinct software description metadata specifications detected in the re
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": 0
},
{
    "name": "repo-gh-workflow-success-ratio",
    "id": "SGA-SF-0001",
    "result-type": "float",
    "sustainability_correlation": 0,
    "description": "Ratio of succeeding workflow runs out of queried workflow runs from GitHub (only
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": 0.94
},
{
    "name": "repo-gh-workflow-succeeding-runs",
    "id": "SGA-SF-0002",
    "result-type": "int",
    "sustainability_correlation": 0,
    "description": "Number of succeeding workflow runs out of queried workflow runs from GitHub (only
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": 100
},
{
    "name": "repo-gh-workflow-failing-runs",
    "id": "SGA-SF-0003",
    "result-type": "int",
    "sustainability_correlation": 0,
    "description": "Number of failing workflow runs out of queried workflow runs from GitHub (only ap
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": 94
},
{
    "name": "repo-gh-workflow-queried-total",
    "id": "SGA-SF-0004",
    "result-type": "int",
    "sustainability_correlation": 0,
    "description": "Total number of workflow runs from GitHub (only applies to GitHub hosted reposito
```

```
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": 6
},
{
    "name": "repo-code-coverage-percent",
    "id": "SGA-SF-0005",
    "result-type": "float",
    "sustainability_correlation": 0,
    "description": "Percentage of code coverage for repository given detected code coverage data.",
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": null
},
{
    "name": "repo-date-of-last-coverage-run",
    "id": "SGA-SF-0006",
    "result-type": "str",
    "sustainability_correlation": 0,
    "description": "Date of code coverage run for repository given detected code coverage data.",
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": null
},
{
    "name": "repo-days-between-last-coverage-run-latest-commit",
    "id": "SGA-SF-0007",
    "result-type": "int",
    "sustainability_correlation": 0,
    "description": "Days between last coverage run date and latest commit date.",
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": null
},
{
    "name": "repo-code-coverage-total-lines",
    "id": "SGA-SF-0008",
    "result-type": "int",
    "sustainability_correlation": 0,
    "description": "Total lines of code used for code coverage within repository given detected code
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": null
},
{
    "name": "repo-code-coverage-executed-lines",
    "id": "SGA-SF-0009",
    "result-type": "int",
    "sustainability_correlation": 0,
    "description": "Total lines covered code within repository given detected code coverage data.",
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": null
},
{
    "name": "repo-agg-info-entropy",
    "id": "SGA-VS-0001",
    "result-type": "float",
    "sustainability_correlation": 0,
    "description": "Aggregated information entropy for all files within a repository given a range be
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": 0.025880455122344243
},
{
    "name": "repo-file-info-entropy",
    "id": "SGA-VS-0002",
```

```
"result-type": "dict",
"sustainability_correlation": 0,
"description": "File-level information entropy for all files within a repository given a range be
"correction_guidance": null,
"fix_why": null,
"fix_how": null,
"result": {
    ".alexignore": 0.0011055470785092086,
    ".github/CODEOWNERS": 0.0049796891155160505,
    ".github/ISSUE_TEMPLATE/bug.yml": 0.027175053398631147,
    ".github/ISSUE_TEMPLATE/config.yml": 0.0011055470785092086,
    ".github/ISSUE_TEMPLATE/feature.yml": 0.024653430173668486,
    ".github/PULL_REQUEST_TEMPLATE.md": 0.01443714719712515,
    ".github/actionlint.yaml": 0.003354624554469599,
    ".github/actions/install-node-env/action.yml": 0.006139587650605865,
    ".github/actions/install-python-env/action.yml": 0.011139852384545063,
    ".github/dependabot.yml": 0.01970207329491403,
    ".github/release-drafter.yml": 0.008709535589062106,
    ".github/workflows/deploy-book.yml": 0.01571008680312421,
    ".github/workflows/draft-release.yml": 0.009416187033173637,
    ".github/workflows/pre-commit-checks.yml": 0.011814052618545701,
    ".github/workflows/publish-pypi.yml": 0.010799642942751366,
    ".github/workflows/pytest-tests.yml": 0.018798632189703908,
    ".github/workflows/test-links.yml": 0.008709535589062106,
    ".gitignore": 0.04489833868545701,
    ".linkcheckerrc.ini": 0.007261454028768062,
    ".pre-commit-config.yaml": 0.05195148489544542,
    ".pre-commit-hooks.yaml": 0.00537109375,
    ".vale.ini": 0.006139587650605865,
    "CITATION.cff": 0.06002995604201093,
    "CODE_OF_CONDUCT.md": 0.0015869140625,
    "CONTRIBUTING.md": 0.0015869140625,
    "LICENSE": 0.011139852384545063,
    "LICENSE.txt": 0.011139852384545063,
    "MANIFEST.in": 0.021480620725612565,
    "README.md": 0.049868807274488496,
    "coverage.xml": 0.34299023335435974,
    "docs/readme.md": 0.0015869140625,
    "media/coverage-badge.svg": 0.0005934636434212709,
    "pa11y.json": 0.003771146647370168,
    "package-lock.json": 0.117026667747954,
    "package.json": 0.002494920736863972,
    "pyproject.toml": 0.059150916088276655,
    "scripts/update_coverage_badge.py": 0.028276512979719795,
    "src/almanack/__init__.py": 0.013139444515869458,
    "src/almanack/batch_processing.py": 0.06414732687283109,
    "src/almanack/book.py": 0.024369315269627673,
    "src/almanack/cli.py": 0.08427816738096243,
    "src/almanack/git.py": 0.10899666845981895,
    "src/almanack/metrics/data.py": 0.3034982038035343,
    "src/almanack/metrics/entropy/calculate_entropy.py": 0.08735404403121295,
    "src/almanack/metrics/entropy/processing_repositories.py": 0.02351196194840558,
    "src/almanack/metrics/garden_lattice/connectedness.py": 0.13028707281638352,
    "src/almanack/metrics/garden_lattice/practicality.py": 0.03595932061635056,
    "src/almanack/metrics/garden_lattice/understanding.py": 0.020001058935756794,
    "src/almanack/metrics/metrics.yml": 0.2118712938810194,
    "src/almanack/metrics/notebooks.py": 0.07026001190916965,
    "src/almanack/metrics/programming_extensions.yml": 0.016649980598861358,
    "src/almanack/metrics/remote.py": 0.052181238080585494,
    "src/almanack/reporting/report.py": 0.0374905854741828,
    "src/book/_config.yml": 0.01910088006373583,
    "src/book/_ext/__init__.py": 0.0,
    "src/book/_ext/gen_check_pages.py": 0.032312444299098436,
    "src/book/_posters/2025/_extensions/quarto-ext/poster/_extension.yml": 0.004582940432061277,
    "src/book/_posters/2025/_extensions/quarto-ext/poster/typst-show.typ": 0.02521922509752699,
    "src/book/_posters/2025/_extensions/quarto-ext/poster/typst-template.typ": 0.050566052524731
    "src/book/_posters/2025/almanack-2025-poster.pdf": 0.0,
    "src/book/_posters/2025/images/almanack-handbook.png": 0.0,
    "src/book/_posters/2025/images/almanack-notebook.png": 0.0,
    "src/book/_posters/2025/images/almanack-package-and-handbook.png": 0.0,
    "src/book/_posters/2025/images/almanack-package.png": 0.0,
    "src/book/_posters/2025/images/bssw-logo-w-background.png": 0.0,
    "src/book/_posters/2025/images/cu-anschutz-short.png": 0.0,
    "src/book/_posters/2025/images/dbmi.png": 0.0,
```

```
"src/book/_posters/2025/images/entropy-for-repos.png": 0.0,
"src/book/_posters/2025/images/forest_modified.png": 0.0,
"src/book/_posters/2025/images/header-combined-images.png": 0.0,
"src/book/_posters/2025/images/roadmap.png": 0.0,
"src/book/_posters/2025/images/seed-bank-entropy-pubmed.png": 0.0,
"src/book/_posters/2025/images/sga-qr-text.png": 0.0,
"src/book/_posters/2025/images/sga-qr.png": 0.0,
"src/book/_posters/2025/images/software-lifecycle.png": 0.0,
"src/book/_posters/2025/images/spacer.png": 0.0,
"src/book/_posters/2025/images/sustainable-horizons-institute-logo.png": 0.0,
"src/book/_posters/2025/images/title-text.png": 0.0,
"src/book/_posters/2025/poster.qmd": 0.03875314440948013,
"src/book/_posters/2025/readme.md": 0.02059590361834423,
"src/book/_static/OSSci Monthly Call Jan 2025 - Software_Gardening_Almanack.odp": 0.0,
"src/book/_static/OSSci Monthly Call Jan 2025 - Software_Gardening_Almanack.pdf": 0.0,
"src/book/_static/custom.css": 0.009064195939365433,
"src/book/_templates/check_template.md.j2": 0.004582940432061277,
"src/book/_toc.yml": 0.01443714719712515,
"src/book/assets/640px-Forgard2-003.gif": 0.0,
"src/book/assets/640px-Rundes_Fenster_mit_Gitter.jpeg": 0.0,
"src/book/assets/Sundial_2916_HDR.jpeg": 0.0,
"src/book/assets/almanack-influencing-software.png": 0.0,
"src/book/assets/garden-lattice-understanding-transfer.png": 0.0,
"src/book/assets/software-gardening-almanack-logo.png": 0.0,
"src/book/assets/software-gardening-logo.png": 0.0,
"src/book/assets/software-lifecycle.png": 0.0,
"src/book/assets/xkcd_dependency.png": 0.0,
"src/book/favicon.png": 0.0,
"src/book/garden-circle/contributing.md": 0.052868558765389095,
"src/book/garden-circle/garden-circle.md": 0.002494920736863972,
"src/book/garden-circle/garden-map.md": 0.016338027295397534,
"src/book/garden-circle/package-api.md": 0.03723662206022836,
"src/book/garden-circle/pavilion.md": 0.013466340410651312,
"src/book/garden-lattice/garden-lattice.md": 0.01571008680312421,
"src/book/garden-lattice/understanding.md": 0.03672721111625638,
"src/book/introduction.md": 0.04489833868545701,
"src/book/references.bib": 0.07818855471704941,
"src/book/seed-bank/almanack-example/almanack-example.ipynb": 0.28285541821882226,
"src/book/seed-bank/pubmed-github-repositories/gather-pubmed-repos/generate_data.py": 0.05423
"src/book/seed-bank/pubmed-github-repositories/gather-pubmed-repos/generate_github_enriched_d
"src/book/seed-bank/pubmed-github-repositories/gather-pubmed-repos/pubmed_github_links.parque
"src/book/seed-bank/pubmed-github-repositories/gather-pubmed-repos/pubmed_github_links_with_d
"src/book/seed-bank/pubmed-github-repositories/images/cdf_repository_inactivity_by_last_commi
"src/book/seed-bank/pubmed-github-repositories/images/primary-language-counts.png": 0.0,
"src/book/seed-bank/pubmed-github-repositories/images/pubmed-lines-of-code-and-time.png": 0.0
"src/book/seed-bank/pubmed-github-repositories/images/pubmed-stars-and-forks.png": 0.0,
"src/book/seed-bank/pubmed-github-repositories/images/pubmed-stars-and-open-issues.png": 0.0,
"src/book/seed-bank/pubmed-github-repositories/images/software-information-entropy-forks.png"
"src/book/seed-bank/pubmed-github-repositories/images/software-information-entropy-gh-stars.p
"src/book/seed-bank/pubmed-github-repositories/images/software-information-entropy-open-issue
"src/book/seed-bank/pubmed-github-repositories/images/software-information-entropy-top-5-lang
"src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_1.parquet":
"src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_10.parquet":
"src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_11.parquet":
"src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_12.parquet":
"src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_13.parquet":
"src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_14.parquet":
"src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_15.parquet":
"src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_16.parquet":
"src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_17.parquet":
"src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_18.parquet":
"src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_19.parquet":
"src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_2.parquet":
"src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_20.parquet":
"src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_21.parquet":
"src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_22.parquet":
"src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_23.parquet":
"src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_24.parquet":
"src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_25.parquet":
"src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_26.parquet":
"src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_27.parquet":
"src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_28.parquet":
"src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_29.parquet":
"src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_3.parquet":
```

```
        "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_30.parquet":
        "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_31.parquet":
        "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_32.parquet":
        "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_33.parquet":
        "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_34.parquet":
        "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_35.parquet":
        "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_36.parquet":
        "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_37.parquet":
        "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_38.parquet":
        "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_39.parquet":
        "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_4.parquet":
        "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_40.parquet":
        "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_41.parquet":
        "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_42.parquet":
        "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_43.parquet":
        "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_44.parquet":
        "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_45.parquet":
        "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_46.parquet":
        "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_47.parquet":
        "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_48.parquet":
        "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_49.parquet":
        "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_5.parquet":
        "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_6.parquet":
        "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_7.parquet":
        "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_8.parquet":
        "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_9.parquet":
        "src/book/seed-bank/pubmed-github-repositories/understanding-pubmed-github-repos.ipynb": 0.15
        "src/book/seed-bank/seed-bank.md": 0.002494920736863972,
        "src/book/software-forest/software-forest.md": 0.01602473970861082,
        "src/book/verdant-sundial/verdant-sundial.md": 0.016338027295397534,
        "styles/config/vocabularies/almanack/accept.txt": 0.012810821801146208,
        "tests/__init__.py": 0.0,
        "tests/conftest.py": 0.05149099830795683,
        "tests/data/almanack/coverage/python/coverage.json": 0.0005934636434212709,
        "tests/data/almanack/coverage/python/coverage.lcov": 0.18782556198306513,
        "tests/data/almanack/repo_setup/create_repo.py": 0.07150063736942495,
        "tests/data/almanack/repo_setup/insert_code.py": 0.01727000168884462,
        "tests/data/jupyter-book/ordered-nb.ipynb": 0.04344748534733697,
        "tests/data/jupyter-book/sandbox.md": 0.006891251628073669,
        "tests/data/jupyter-book/unordered-nb.ipynb": 0.04344748534733697,
        "tests/metrics/test_calculate_entropy.py": 0.07108797392377063,
        "tests/metrics/test_data.py": 0.281557681162887,
        "tests/metrics/test_garden_lattice.py": 0.06264129748270111,
        "tests/metrics/test_remote.py": 0.011814052618545701,
        "tests/test_almanack.py": 0.013466340410651312,
        "tests/test_batch.py": 0.045857710687013484,
        "tests/test_build.py": 0.02408437352963596,
        "tests/test_cli.py": 0.052181238080585494,
        "tests/test_git.py": 0.10917030478732884,
        "tests/test_notebooks.py": 0.08925453229089084,
        "tests/utils.py": 0.013466340410651312,
        "uv.lock": 0.38628234006896384
    }
},
{
    "name": "repo-agg-history-complexity-decay",
    "id": "SGA-VS-0003",
    "result-type": "float",
    "sustainability_correlation": 0,
    "description": "Aggregated history complexity with decay (HCM_1d) for all files within a reposito
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
    "result": 0.013469060651110525
},
{
    "name": "repo-file-history-complexity-decay",
    "id": "SGA-VS-0004",
    "result-type": "dict",
    "sustainability_correlation": 0,
    "description": "File-level history complexity with decay (HCM_1d) over a range between two commit
    "correction_guidance": null,
    "fix_why": null,
    "fix_how": null,
```

```
"result": {
    ".github/ISSUE_TEMPLATE/bug.yml": 0.0,
    "LICENSE.txt": 0.0,
    "src/book/_posters/2025/images/sga-qr.png": 0.0,
    "src/book/_toc.yml": 0.0,
    "src/book/_posters/2025/images/almanack-notebook.png": 0.0,
    "tests/utils.py": 0.0,
    "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_45.parquet":
    "src/book/garden-circle/garden-circle.md": 0.0,
    "src/book/_posters/2025/images/forest_modified.png": 0.0,
    "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_20.parquet":
    "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_5.parquet":
    "src/almanack/metrics/remote.py": 6.535548254983671e-301,
    "styles/config/vocabularies/almanack/accept.txt": 1.0895391633472122e-116,
    "tests/metrics/test_data.py": 2.347014702131075e-99,
    "src/book/seed-bank/almanack-example/almanack-example.ipynb": 0.0,
    "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_42.parquet":
    "src/book/_posters/2025/images/header-combined-images.png": 0.0,
    "src/book/_posters/2025/images/sustainable-horizons-institute-logo.png": 0.0,
    "tests/metrics/test_remote.py": 1.3389589095172126e-303,
    "src/book/assets/640px-Rundes_Fenster_mit_Gitter.jpeg": 0.0,
    "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_9.parquet":
    "src/book/garden-circle/garden-map.md": 0.0,
    ".pre-commit-config.yaml": 5.441262304207869e-05,
    "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_32.parquet":
    "src/book/_posters/2025/_extensions/quarto-ext/poster/typst-template.typ": 0.0,
    "src/book/_posters/2025/images/bssw-logo-w-background.png": 0.0,
    "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_1.parquet":
    "tests/test_build.py": 2.0012716541157927e-116,
    "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_27.parquet":
    "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_4.parquet":
    "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_41.parquet":
    "src/almanack/batch_processing.py": 1.410378737975496e-99,
    "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_29.parquet":
    "tests/data/almanack/repo_setup/create_repo.py": 0.0,
    "src/book/seed-bank/pubmed-github-repositories/images/software-information-entropy-top-5-lang
    ".github/workflows/pytest-tests.yml": 0.4556795091007162,
    "src/book/seed-bank/pubmed-github-repositories/understanding-pubmed-github-repos.ipynb": 7.61
    "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_40.parquet":
    "src/almanack/metrics/data.py": 5.802594330412838e-25,
    "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_28.parquet":
    ".github/workflows/pre-commit-checks.yml": 0.3231002236579754,
    "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_7.parquet":
    "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_33.parquet":
    "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_49.parquet":
    "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_11.parquet":
    "src/almanack/metrics/garden_lattice/connectedness.py": 2.9697969186545597e-248,
    "tests/data/jupyter-book/sandbox.md": 0.0,
    "src/book/seed-bank/pubmed-github-repositories/images/pubmed-stars-and-open-issues.png": 0.0,
    "src/book/seed-bank/pubmed-github-repositories/images/software-information-entropy-forks.png"
    "tests/test_batch.py": 3.987e-321,
    ".alexignore": 0.0,
    "src/book/_posters/2025/images/software-lifecycle.png": 0.0,
    "src/book/assets/software-lifecycle.png": 0.0,
    "src/book/_templates/check_template.md.j2": 0.0,
    "src/almanack/__init__.py": 9.24988843873215e-116,
    "CITATION.cff": 0.0,
    "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_18.parquet":
    "src/book/garden-circle/contributing.md": 1.2938740535569688e-115,
    ".github/workflows/draft-release.yml": 0.32307784978845333,
    "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_17.parquet":
    "src/almanack/book.py": 2.845892127418932e-116,
    "CONTRIBUTING.md": 0.0,
    "src/almanack/metrics/notebooks.py": 2.700886159803712e-279,
    "LICENSE": 0.0,
    "src/book/_posters/2025/almanack-2025-poster.pdf": 0.0,
    "tests/test_almanack.py": 0.0,
    "src/book/_posters/2025/images/entropy-for-repos.png": 0.0,
    "tests/data/jupyter-book/unordered-nb.ipynb": 1.558882967691099e-284,
    ".vale.ini": 0.0,
    ".github/workflows/deploy-book.yml": 0.32307784978845333,
    ".github/ISSUE_TEMPLATE/config.yml": 0.0,
    "src/book/seed-bank/pubmed-github-repositories/images/primary-language-counts.png": 0.0,
    "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_8.parquet":
```

```
"src/almanack/metrics/entropy/calculate_entropy.py": 3.122744550503043e-253,
"src/book/seed-bank/pubmed-github-repositories/gather-pubmed-repos/generate_github_enriched_c
"src/book/_posters/2025/images/almanack-package-and-handbook.png": 0.0,
"src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_44.parquet":
".linkcheckerrc.ini": 0.0,
"tests/conftest.py": 6.390778039582678e-70,
"src/book/_posters/2025/images/almanack-handbook.png": 0.0,
"package.json": 2.7496168366936856e-70,
"src/book/_posters/2025/images/sga-qr-text.png": 0.0,
"src/book/garden-lattice/understanding.md": 3.945402594226073e-178,
"src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_31.parquet":
"src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_46.parquet":
"src/almanack/git.py": 6.047120876569832e-285,
"src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_21.parquet":
"src/book/_ext/gen_check_pages.py": 0.0,
"src/book/_posters/2025/readme.md": 0.0,
"src/book/_static/OSSci Monthly Call Jan 2025 - Software_Gardening_Almanack.odp": 0.0,
".github/dependabot.yml": 2.741401736225424e-34,
"src/book/assets/software-gardening-almanack-logo.png": 0.0,
"src/book/seed-bank/seed-bank.md": 0.0,
".github/actions/install-node-env/action.yml": 2.0012716541157927e-116,
"src/book/seed-bank/pubmed-github-repositories/gather-pubmed-repos/pubmed_github_links.parque
"src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_36.parquet":
"src/book/_posters/2025/_extensions/quarto-ext/poster/typst-show.typ": 0.0,
"src/book/seed-bank/pubmed-github-repositories/images/cdf_repository_inactivity_by_last_commi
"src/book/_posters/2025/images/cu-anschutz-short.png": 0.0,
"src/book/_posters/2025/images/almanack-package.png": 0.0,
"src/book/favicon.png": 0.0,
"src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_43.parquet":
"src/book/introduction.md": 0.0,
"src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_16.parquet":
"src/book/seed-bank/pubmed-github-repositories/gather-pubmed-repos/generate_data.py": 2.00127
"scripts/update_coverage_badge.py": 4.41764486710759e-115,
"src/book/assets/Sundial_2916_HDR.jpeg": 0.0,
"src/book/verdant-sundial/verdant-sundial.md": 0.0,
"src/book/_posters/2025/images/dbmi.png": 0.0,
"src/book/_posters/2025/images/title-text.png": 0.0,
"src/book/seed-bank/pubmed-github-repositories/gather-pubmed-repos/pubmed_github_links_with_g
"src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_37.parquet":
"src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_2.parquet":
"src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_24.parquet":
"src/almanack/reporting/report.py": 0.0,
"src/almanack/cli.py": 1.410378737975496e-99,
"src/book/assets/almanack-influencing-software.png": 0.0,
"src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_15.parquet":
"src/almanack/metrics/metrics.yml": 4.3898124360526285e-230,
"tests/data/almanack/repo_setup/insert_code.py": 1.0401622071153047e-303,
"src/book/assets/xkcd_dependency.png": 0.0,
"src/almanack/metrics/garden_lattice/understanding.py": 0.0,
"src/book/_ext/__init__.py": 0.0,
"src/book/garden-lattice/garden-lattice.md": 0.0,
"src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_47.parquet":
".github/ISSUE_TEMPLATE/feature.yml": 0.0,
".github/actions/install-python-env/action.yml": 1.686598376020089e-25,
"pyproject.toml": 3.940208764089633e-05,
"src/book/_static/OSSci Monthly Call Jan 2025 - Software_Gardening_Almanack.pdf": 0.0,
".github/release-drafter.yml": 0.0,
"tests/test_cli.py": 1.54e-321,
"src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_19.parquet":
"src/book/_posters/2025/poster.qmd": 0.0,
"src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_34.parquet":
"src/book/seed-bank/pubmed-github-repositories/images/software-information-entropy-open-issue
"tests/test_notebooks.py": 2.3534638584112833e-279,
".github/CODEOWNERS": 0.0,
".github/workflows/publish-pypi.yml": 0.32307784978845333,
".github/PULL_REQUEST_TEMPLATE.md": 0.0,
"src/book/_static/custom.css": 3.6469299630743217e-116,
"src/book/seed-bank/pubmed-github-repositories/images/pubmed-stars-and-forks.png": 0.0,
"tests/metrics/test_garden_lattice.py": 1.808662227091261e-248,
"tests/__init__.py": 0.0,
"src/book/seed-bank/pubmed-github-repositories/images/pubmed-lines-of-code-and-time.png": 0.0
"src/book/software-forest/software-forest.md": 0.0,
"src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_30.parquet":
"src/book/references.bib": 0.0,
```

```
            "src/book/_posters/2025/_extensions/quarto-ext/poster/_extension.yml": 0.0,
            ".github/actionlint.yaml": 0.5283208335737187,
            "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_14.parquet":
            "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_23.parquet":
            "src/book/assets/software-gardening-logo.png": 0.0,
            "tests/data/jupyter-book/ordered-nb.ipynb": 1.558882967691099e-284,
            "tests/data/almanack/coverage/python/coverage.json": 0.0,
            "pa11y.json": 1.0895391633472122e-116,
            "uv.lock": 2.292545449006218e-05,
            "src/book/_config.yml": 2.0012716541157927e-116,
            "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_22.parquet":
            "src/book/garden-circle/package-api.md": 4.45e-322,
            "docs/readme.md": 0.0,
            "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_38.parquet":
            "src/almanack/metrics/entropy/processing_repositories.py": 0.0,
            "src/book/garden-circle/pavilion.md": 5.15836423710197e-116,
            "src/book/_posters/2025/images/seed-bank-entropy-pubmed.png": 0.0,
            "tests/data/almanack/coverage/python/coverage.lcov": 0.0,
            ".pre-commit-hooks.yaml": 5.187026922378882e-280,
            "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_10.parquet":
            "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_25.parquet":
            "CODE_OF_CONDUCT.md": 0.0,
            "src/book/seed-bank/pubmed-github-repositories/images/software-information-entropy-gh-stars.p
            "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_39.parquet":
            "tests/test_git.py": 1.410378737975496e-99,
            "src/book/_posters/2025/images/spacer.png": 0.0,
            "coverage.xml": 7.750419575991968e-25,
            "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_3.parquet":
            "README.md": 3.826508120714108e-176,
            ".gitignore": 7.963546479005618e-286,
            "src/almanack/metrics/garden_lattice/practicality.py": 0.0,
            "src/almanack/metrics/programming_extensions.yml": 1.3804331650884058e-230,
            "src/book/_posters/2025/images/roadmap.png": 0.0,
            "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_26.parquet":
            "tests/metrics/test_calculate_entropy.py": 2.6420533667822448e-253,
            "src/book/assets/640px-Forgard2-003.gif": 0.0,
            "package-lock.json": 1.2934748993286078e-11,
            "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_12.parquet":
            "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_48.parquet":
            "src/book/assets/garden-lattice-understanding-transfer.png": 0.0,
            "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_6.parquet":
            ".github/workflows/test-links.yml": 0.32307784978845333,
            "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_35.parquet":
            "src/book/seed-bank/pubmed-github-repositories/repository_analysis_results/batch_13.parquet":
            "MANIFEST.in": 3.4298886470317697e-115,
            "media/coverage-badge.svg": 1.686598376020089e-25
        }
    }
]
```


```
%%time

# show the same results from the CLI
!almanack table "./almanack"
```

```
Transient HTTP 503 for https://api.openalex.org/works (attempt 1/3). Retrying in 2 seconds...


Transient HTTP 503 for https://api.openalex.org/works (attempt 2/3). Retrying in 4 seconds...


[{"name": "repo-path", "id": "SGA-META-0001", "result-type": "str", "sustainability_correlation": 0, "des


CPU times: user 166 ms, sys: 62.5 ms, total: 229 ms
Wall time: 17.9 s
```

```
%%time

# show the same results from the CLI
!almanack check "./almanack"
```

```
Running Software Gardening Almanack checks.
Datetime: 2026-11-05T15:27:10.038660Z
Almanack version: 0.1.dev1+g39a329bd0
Target repository path: ./almanack


Transient HTTP 503 for https://api.openalex.org/works (attempt 1/3). Retrying in 2 seconds...


Transient HTTP 503 for https://api.openalex.org/works (attempt 2/3). Retrying in 4 seconds...


The following Software Gardening Almanack metrics may be helpful to improve your repository:
┌──────────────┬──────────────────────────────┬──────────────────────────────┐
│ ID           │ Name                         │ Guidance                     │
├──────────────┼──────────────────────────────┼──────────────────────────────┤
│ SGA-GL-0030  │ repo-check-notebook-exec-order │ Consider executing notebook │
│              │                              │ cells in sequential order    │
│              │                              │ to improve reproducibility.  │
└──────────────┴──────────────────────────────┴──────────────────────────────┘
Software Gardening Almanack summary: 90.91% (10/11)


CPU times: user 136 ms, sys: 52.4 ms, total: 188 ms
Wall time: 15 s
```

Undersatanding PubMed GitHub repositories with the Software Gardening Almanack

The content below seeks to better understand a dataset of ~10,000 PubMed article GitHub repositories using the Software Gardening Almanack.

PubMed GitHub repositories are extracted using the PubMed API to query for GitHub links within article abstracts. GitHub data about these repositories is gathered using the GitHub API. The code to perform data extractions may be found under the directory: gather-pubmed-repos .

These repositories are then processed using the Software Gardening Almanack to contextualize how they compare to one another.

Data Extraction

The following section extracts data which includes software entropy and GitHub-derived data. We merge the data to form a table which includes PubMed, GitHub, and Almanack software entropy information on the repositories.

```
import pathlib

import numpy as np
import pandas as pd
import plotly.express as px
import plotly.io as pio
from IPython.display import Image, display

from almanack import process_repositories_batch

# set plotly default theme
pio.templates.default = "plotly_white"

# set almanack data directory
ALMANACK_DATA_DIR = "repository_analysis_results"
```

```
# if we don't already have a data dir, batch process almanack data
if not pathlib.Path(ALMANACK_DATA_DIR).is_dir():
    process_repositories_batch(
        # read pre-collected pubmeb github repos
        repo_urls=pd.read_parquet(
            path="gather-pubmed-repos/pubmed_github_links_with_github_data.parquet",
            columns=["github_link"],
        )
        .dropna()
        .github_link.tolist()[1:20],
        output_path=ALMANACK_DATA_DIR,
        split_batches=True,
        batch_size=500,
        max_workers=8,
        collect_dataframe=False,
        show_repo_progress=False,
        show_batch_progress=True,
    )

# read the dataset as a concenated dataframe from parquet chunks
df = pd.concat(
    [
        pd.read_parquet(pq_file)
        for pq_file in pathlib.Path(ALMANACK_DATA_DIR).glob("*.parquet")
    ],
    ignore_index=True,
    sort=False,
)
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 9631 entries, 0 to 9630
Data columns (total 62 columns):
 #   Column                                                  Non-Null Count  Dtype
---  ------                                                  --------------  -----
 0   Repository URL                                          9631 non-null   str
 1   repo-path                                               9631 non-null   str
 2   repo-commits                                            9587 non-null   float64
 3   repo-file-count                                         9587 non-null   float64
 4   repo-commit-time-range                                  9631 non-null   str
 5   repo-days-of-development                                9587 non-null   float64
 6   repo-commits-per-day                                    9587 non-null   float64
 7   almanack-table-datetime                                 9631 non-null   str
 8   almanack-version                                        9631 non-null   str
 9   repo-primary-language                                   5789 non-null   str
 10  repo-primary-license                                    3503 non-null   str
 11  repo-doi                                                125 non-null    object
 12  repo-doi-publication-date                               107 non-null    object
 13  repo-doi-fwci                                           46 non-null     object
 14  repo-doi-is-not-retracted                               75 non-null     object
 15  repo-doi-grants-count                                   84 non-null     object
 16  repo-doi-grants                                         0 non-null      object
 17  repo-includes-readme                                    9587 non-null   float64
 18  repo-includes-contributing                              9587 non-null   float64
 19  repo-includes-code-of-conduct                           9587 non-null   float64
 20  repo-includes-license                                   9587 non-null   float64
 21  repo-is-citable                                         9587 non-null   float64
 22  repo-default-branch-not-master                          9587 non-null   float64
 23  repo-includes-common-docs                               9587 non-null   float64
 24  repo-unique-contributors                                9587 non-null   float64
 25  repo-unique-contributors-past-year                      9587 non-null   float64
 26  repo-unique-contributors-past-182-days                  9587 non-null   float64
 27  repo-tags-count                                         9587 non-null   float64
 28  repo-tags-count-past-year                               9587 non-null   float64
 29  repo-tags-count-past-182-days                           9587 non-null   float64
 30  repo-stargazers-count                                   6024 non-null   float64
 31  repo-uses-issues                                        6024 non-null   float64
 32  repo-issues-open-count                                  6024 non-null   float64
 33  repo-pull-requests-enabled                              6024 non-null   float64
 34  repo-forks-count                                        6024 non-null   float64
 35  repo-subscribers-count                                  6024 non-null   float64
 36  repo-packages-ecosystems-count                          9587 non-null   float64
 37  repo-packages-versions-count                            9587 non-null   float64
 38  repo-social-media-platforms-count                       9295 non-null   float64
 39  repo-doi-valid-format                                   90 non-null     object
 40  repo-doi-https-resolvable                               84 non-null     object
 41  repo-doi-cited-by-count                                 75 non-null     object
 42  repo-days-between-doi-publication-date-and-latest-commit  75 non-null     object
 43  repo-gh-workflow-success-ratio                          463 non-null    float64
 44  repo-gh-workflow-succeeding-runs                        463 non-null    float64
 45  repo-gh-workflow-failing-runs                           463 non-null    float64
 46  repo-gh-workflow-queried-total                          463 non-null    float64
 47  repo-code-coverage-percent                              0 non-null      object
 48  repo-date-of-last-coverage-run                          0 non-null      object
 49  repo-days-between-last-coverage-run-latest-commit       0 non-null      object
 50  repo-code-coverage-total-lines                          0 non-null      object
 51  repo-code-coverage-executed-lines                       0 non-null      object
 52  repo-agg-info-entropy                                   9587 non-null   float64
 53  repo-almanack-score_json                                9631 non-null   str
 54  repo-packages-ecosystems_json                           9631 non-null   str
 55  repo-social-media-platforms_json                        9631 non-null   str
 56  checks_total                                            9587 non-null   float64
 57  checks_passed                                           9587 non-null   float64
 58  checks_pct                                              9587 non-null   float64
 59  repo-social-media-platforms                             0 non-null      float64
 60  almanack_error                                          6000 non-null   str
 61  repo-doi-grants_json                                    4200 non-null   str
dtypes: float64(35), object(15), str(12)
memory usage: 6.9+ MB
```

How many repositories are dormant?

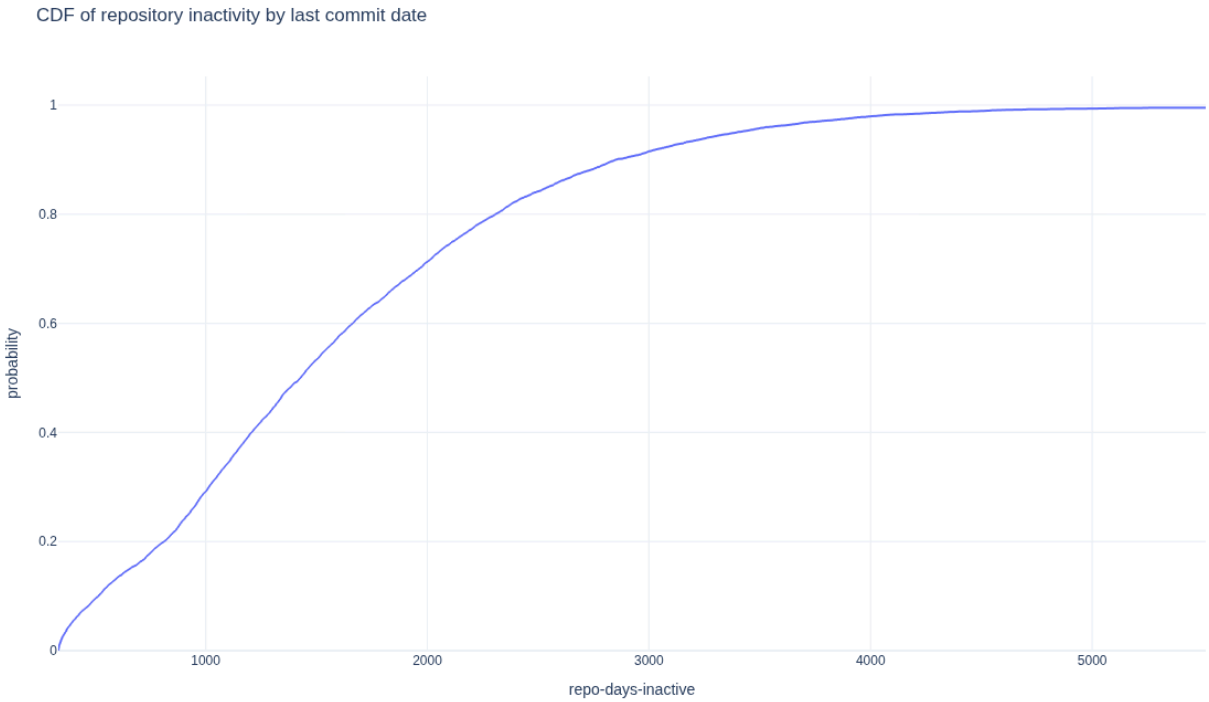
Software must evolve continuously to remain stable, useful, and aligned with current needs. The following section explores how repository dormancy appears within the dataset based on the date of most recent commit. Sustained, incremental updates help prevent this stagnation and ensure that a project continues to serve its users effectively.

```
# cleanup: drop columns or rows whose values are all empty
df = df.dropna(axis=1, how="all")
df = df.dropna(axis=0, how="all")
```

```
# gather repo commit start and end dates
df[["repo-commit-start-date", "repo-commit-end-date"]] = df[
    "repo-commit-time-range"
].str.extract(r"\('([^']+)', '([^']+)'\)")
# turn these into datetimes for datetime calcs
df[["repo-commit-start-date", "repo-commit-end-date"]] = df[
    ["repo-commit-start-date", "repo-commit-end-date"]
].apply(pd.to_datetime)
# set a cuttoff of 1 year for dormancy
cutoff = pd.Timestamp.now() - pd.DateOffset(years=1)
df["dormant"] = df["repo-commit-end-date"] <= cutoff
# show how many dormant repos there are in this dataset
df["dormant"].value_counts()
```

```
dormant
True     9278
False     353
Name: count, dtype: int64
```

```
df["repo-days-of-existence"] = (
    pd.Timestamp.now() - df["repo-commit-start-date"]
).dt.days
df["repo-days-inactive"] = (pd.Timestamp.now() - df["repo-commit-end-date"]).dt.days
fig = px.ecdf(
    df,
    x="repo-days-inactive",
    title="CDF of repository inactivity by last commit date",
    width=1200,
    height=700,
)
fig.write_image(
    image_file := "images/cdf_repository_inactivity_by_last_commit_date.png"
)
display(Image(filename=image_file))
```



What languages are used within PubMed article repositories?

The following section observes the top 10 languages which are used in repositories from the dataset. Primary language is determined as the language which has the most lines of code within a repository.

```
# Count dormant & active repos per language
lang_status = (
    df.groupby(["repo-primary-language", "dormant"]).size().reset_index(name="Count")
)

# Map boolean to readable labels
lang_status["Status"] = lang_status["dormant"].map(
    {
        True: "Dormant (no commits >= 1 year)",
        False: "Active (< 1 year since last commit)",
    }
)

# Total repos per language
total_counts = lang_status.groupby("repo-primary-language")["Count"].sum()

# Top 10 languages by total repo count (sorted)
top_langs = total_counts.sort_values().tail(10)

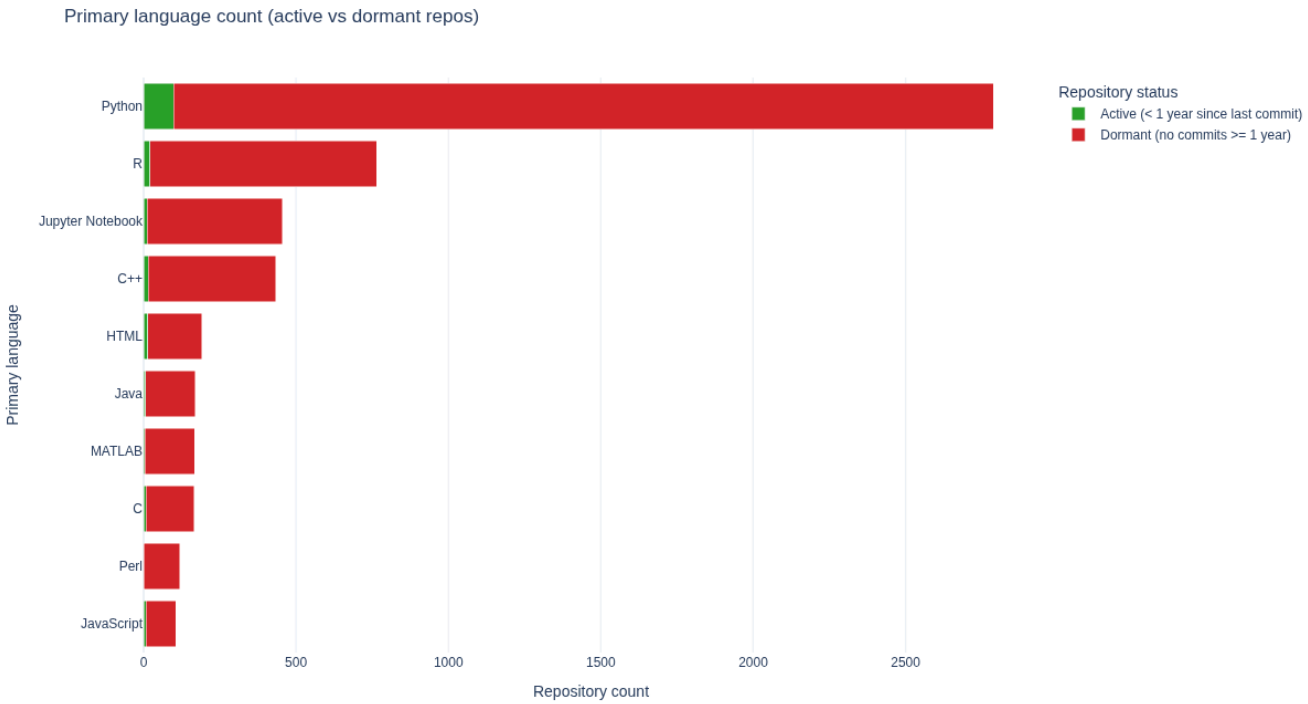
# Filter to top languages
plot_data = lang_status[
    lang_status["repo-primary-language"].isin(top_langs.index)
].copy()

# Ensure y-axis follows the sorted order by total count
lang_order = top_langs.index.tolist()
plot_data["repo-primary-language"] = pd.Categorical(
    plot_data["repo-primary-language"],
    categories=lang_order,
    ordered=True,
)

# Stacked horizontal bar chart: Active (green) + Dormant (red)
fig = px.bar(
    plot_data,
    y="repo-primary-language",
    x="Count",
    color="Status",
    orientation="h",
    color_discrete_map={
        "Active (< 1 year since last commit)": "#2ca02c",
        "Dormant (no commits >= 1 year)": "#d62728",
    },
    width=1200,
    height=700,
    title="Primary language count (active vs dormant repos)",
)

fig.update_layout(
    barmode="stack",
    xaxis_title="Repository count",
    yaxis_title="Primary language",
    legend_title="Repository status",
    yaxis=dict(categoryorder="array", categoryarray=lang_order),
)


fig.write_image(image_file := "images/primary-language-counts.png")
display(Image(filename=image_file))
```



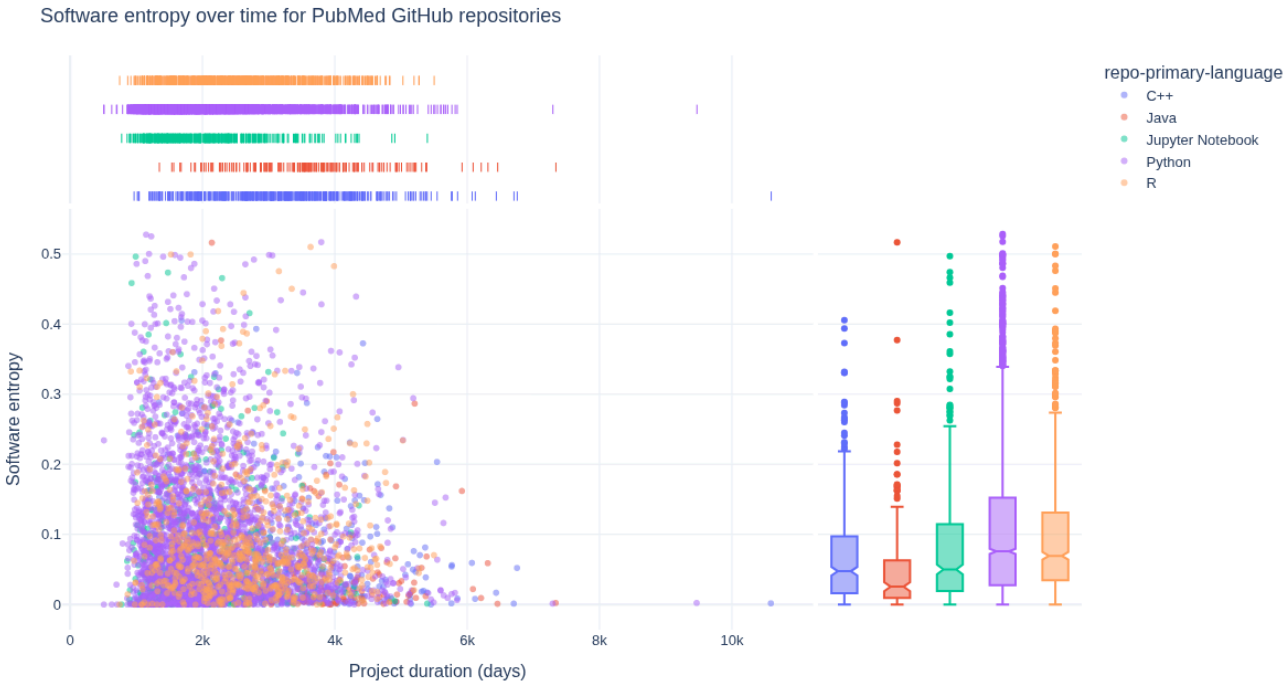
How is software entropy different across primary languages?

The following section explores how software entropy manifests differently across different primary languages for repositories.

```
language_grouped_data = (
    df.groupby(["repo-primary-language"]).size().reset_index(name="Count")
)

fig = px.scatter(
    df[
        df["repo-primary-language"].isin(
            language_grouped_data
            # exclude HTML to focus on formal programming languages
            .loc[
                ~language_grouped_data["repo-primary-language"].str.contains(
                    "html", case=False, na=False
                )
            ].sort_values(by="Count")[-5:]["repo-primary-language"]
        )
    ].sort_values(by="repo-primary-language"),
    x="repo-days-of-existence",
    y="repo-agg-info-entropy",
    hover_data=["Repository URL"],
    width=1200,
    height=700,
    title="Software entropy over time for PubMed GitHub repositories",
    marginal_x="rug",
    marginal_y="box",
    opacity=0.5,
    color="repo-primary-language",
)

fig.update_layout(
    font=dict(size=13),
    title={"yref": "container", "y": 0.9, "yanchor": "bottom"},
    xaxis_title="Project duration (days)",
    yaxis_title="Software entropy",
)
fig.write_image(image_file := "images/software-information-entropy-top-5-langs.png")
display(Image(filename=image_file))
```

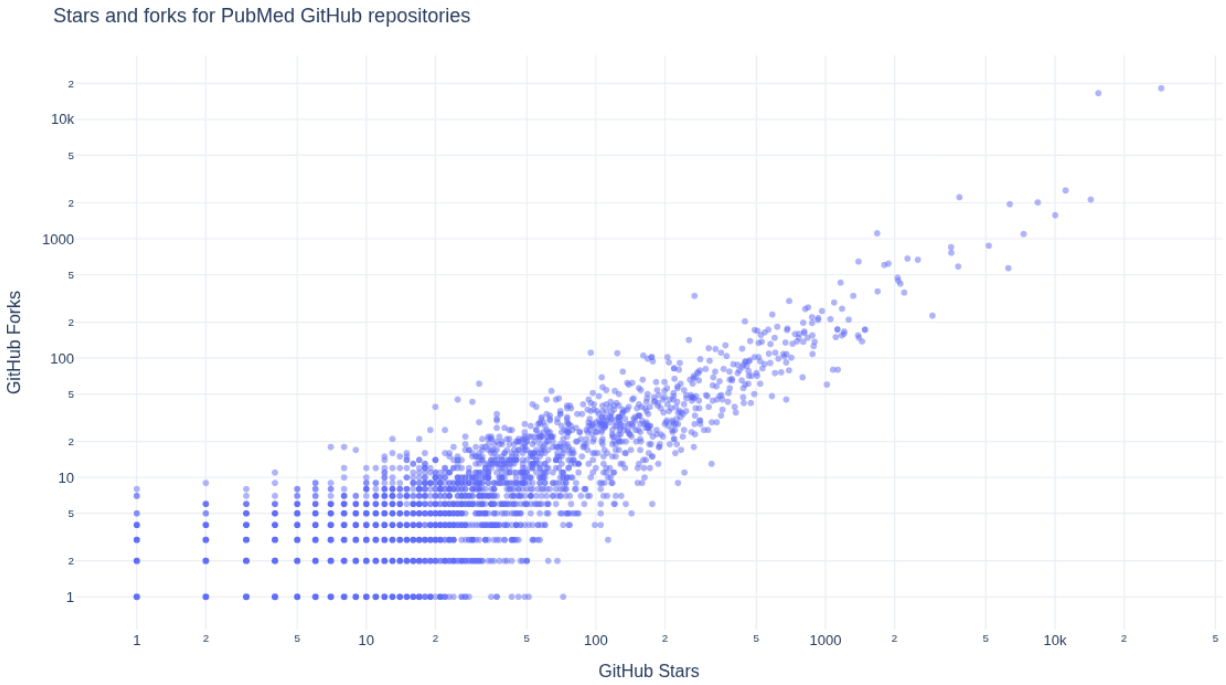


What is the relationship between GitHub Stars and Forks for repositories?

We next explore how GitHub Stars and Forks are related within the repositories.

```
fig = px.scatter(
    df,
    x="repo-stargazers-count",
    y="repo-forks-count",
    hover_data=["Repository URL"],
    width=1200,
    height=700,
    title="Stars and forks for PubMed GitHub repositories",
    opacity=0.5,
    log_y=True,
    log_x=True,
)

fig.update_layout(
    font=dict(size=13),
    title={"yref": "container", "y": 0.9, "yanchor": "bottom"},
    xaxis_title="GitHub Stars",
    yaxis_title="GitHub Forks",
)
fig.write_image(image_file := "images/pubmed-stars-and-forks.png")
display(Image(filename=image_file))
```



How do GitHub Stars, software entropy, and time relate?

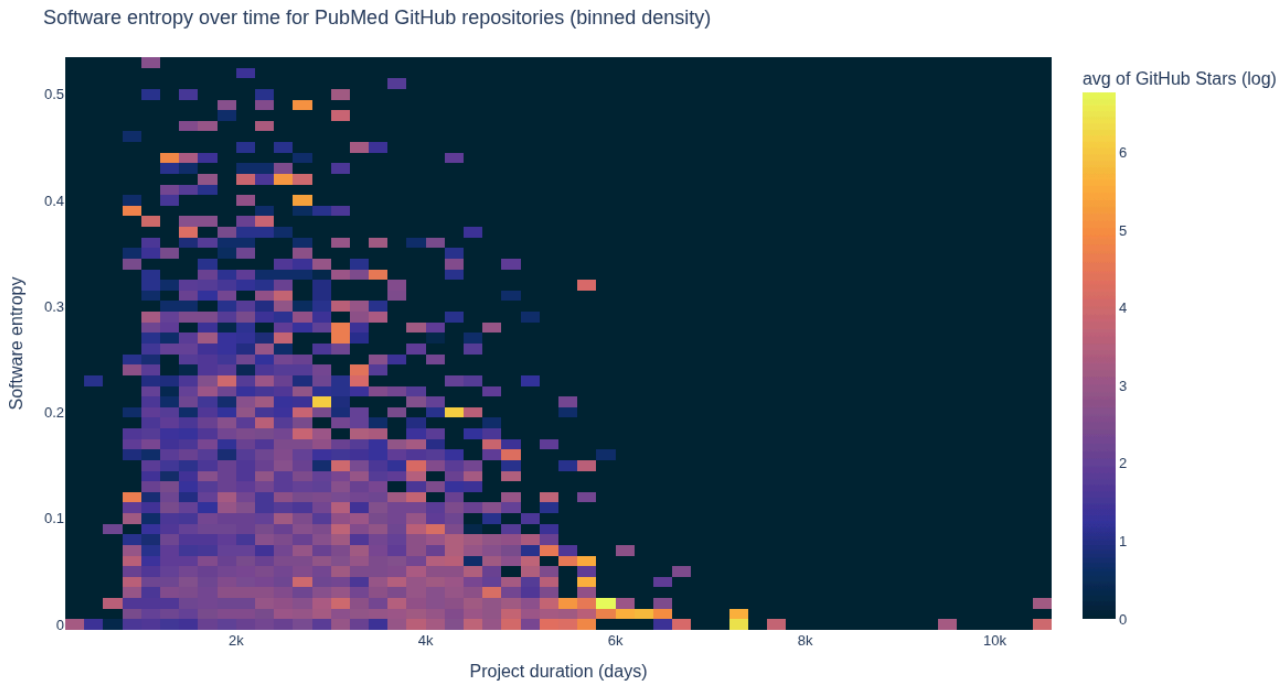
The next section explores how GitHub Stars, software entropy, and time relate to one another.

```
df["GitHub Stars (log)"] = np.log(
    df["repo-stargazers-count"].apply(lambda x: x if x > 0 else None)  # avoid log(0)
)

df_plot = df.dropna(subset=["GitHub Stars (log)"])

fig = px.density_heatmap(
    df_plot,
    x="repo-days-of-existence",
    y="repo-agg-info-entropy",
    z="GitHub Stars (log)",  # aggregate value per bin
    histfunc="avg",  # mean log stars per bin (could use "sum" or "count")
    nbinsx=60,
    nbinsy=60,
    color_continuous_scale=px.colors.sequential.thermal,
    hover_data=["Repository URL"],
    width=1200,
    height=700,
    title="Software entropy over time for PubMed GitHub repositories (binned density)",
)

fig.update_layout(
    font=dict(size=13),
    title={"yref": "container", "y": 0.9, "yanchor": "bottom"},
    xaxis_title="Project duration (days)",
    yaxis_title="Software entropy",
)
fig.write_image(image_file := "images/software-information-entropy-gh-stars.png")
display(Image(filename=image_file))
```



How do GitHub Forks, software entropy, and time relate?

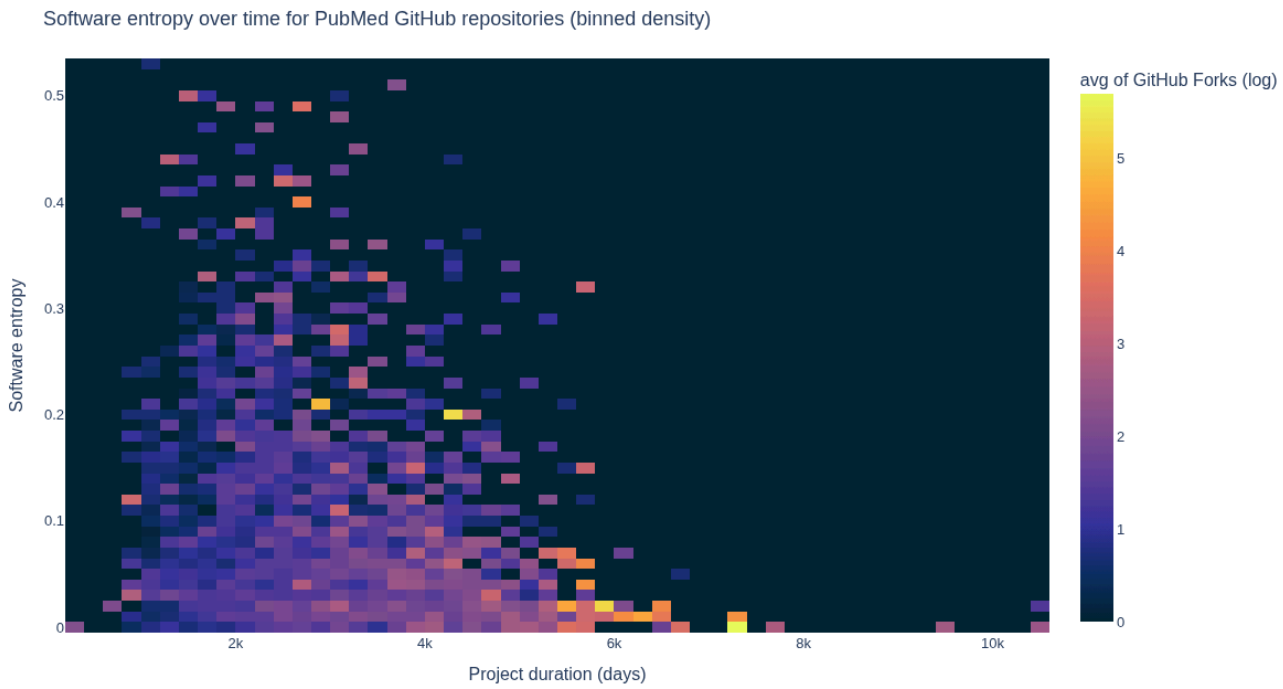
The next section explores how GitHub Forks, software entropy, and time relate to one another.

```
df["GitHub Forks (log)"] = np.log(
    df["repo-forks-count"].apply(
        # move 0's to None to avoid log(0)
        lambda x: x if x > 0 else None
    )
)

# Drop rows with missing values in the relevant columns
df_plot = df.dropna(
    subset=["GitHub Forks (log)", "repo-days-of-existence", "repo-agg-info-entropy"]
)

fig = px.density_heatmap(
    df_plot,
    x="repo-days-of-existence",
    y="repo-agg-info-entropy",
    z="GitHub Forks (log)",  # value to aggregate per bin
    histfunc="avg",  # mean log forks per bin (can use "sum" or "count")
    nbinsx=60,
    nbinsy=60,
    color_continuous_scale=px.colors.sequential.thermal,
    width=1200,
    height=700,
    title="Software entropy over time for PubMed GitHub repositories (binned density)",
)

fig.update_layout(
    font=dict(size=13),
    title={"yref": "container", "y": 0.9, "yanchor": "bottom"},
    xaxis_title="Project duration (days)",
    yaxis_title="Software entropy",
)
fig.write_image(image_file := "images/software-information-entropy-forks.png")
display(Image(filename=image_file))
```



What is the relationship between GitHub Stars and Open Issues

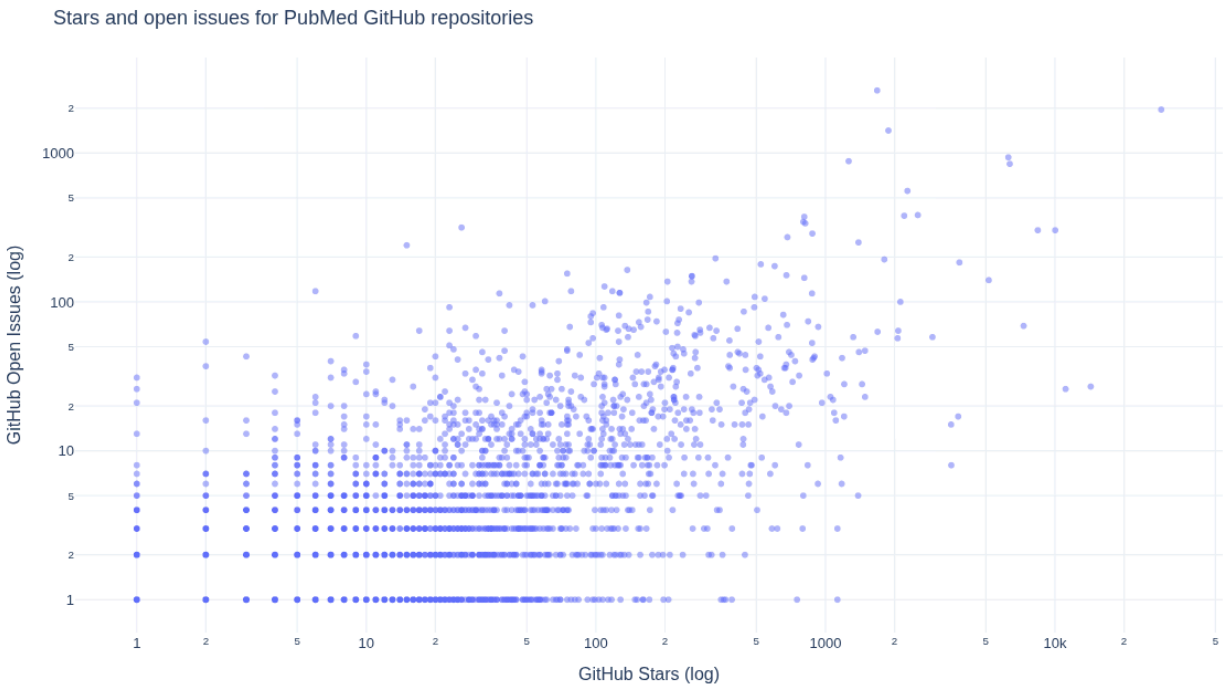
for the repositories?

Below we explore how GitHub Stars and Open Issues are related for the repositories.

```
df["GitHub Open Issues (log)"] = np.log(
    df["repo-issues-open-count"].apply(
        # move 0's to None to avoid divide by 0
        lambda x: x if x > 0 else None
    )
)

fig = px.scatter(
    df,
    x="repo-stargazers-count",
    y="repo-issues-open-count",
    hover_data=["Repository URL"],
    width=1200,
    height=700,
    title="Stars and open issues for PubMed GitHub repositories",
    opacity=0.5,
    log_y=True,
    log_x=True,
)

fig.update_layout(
    font=dict(size=13),
    title={"yref": "container", "y": 0.9, "yanchor": "bottom"},
    xaxis_title="GitHub Stars (log)",
    yaxis_title="GitHub Open Issues (log)",
)
fig.write_image(image_file := "images/pubmed-stars-and-open-issues.png")
display(Image(filename=image_file))
```



How do software entropy, time, and GitHub issues relate to one another?

The next section visualizes how software entropy, time, and GitHub issues relate to one another.

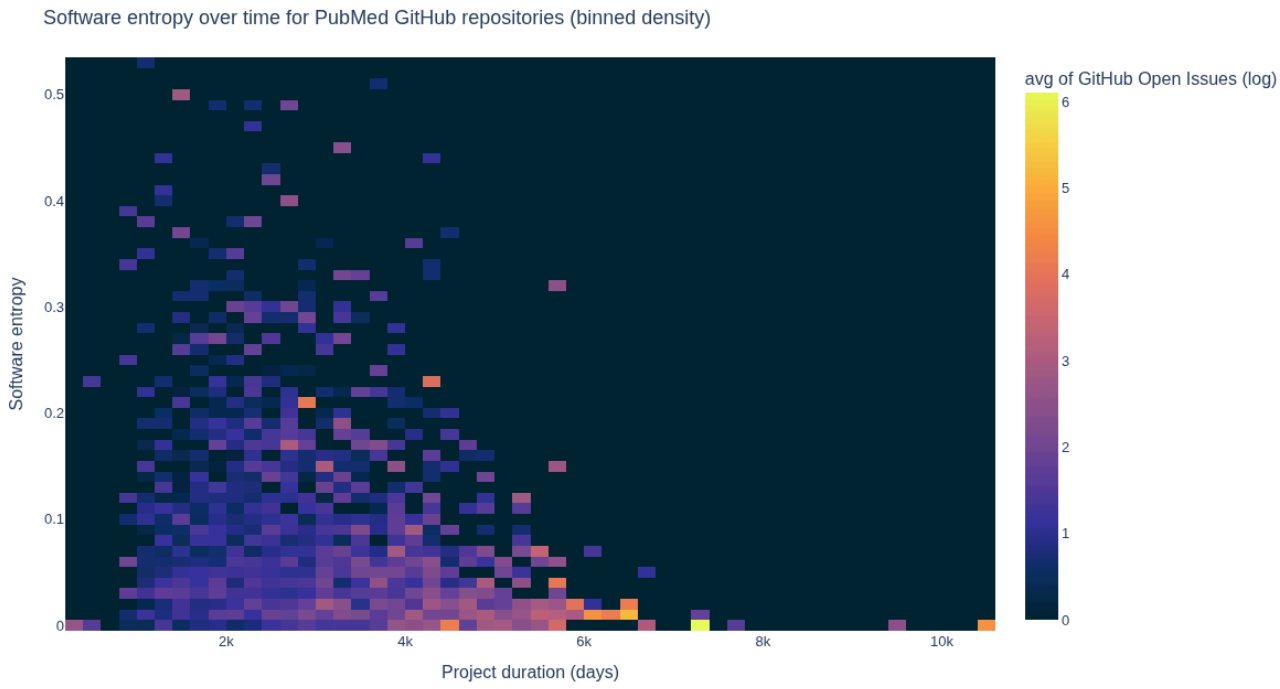
```
df["GitHub Open Issues (log)"] = np.log(
    df["repo-issues-open-count"].apply(
        # move 0's to None to avoid log(0)
        lambda x: x if x > 0 else None
    )
)

# Drop rows with missing values in the relevant columns
df_plot = df.dropna(
    subset=[
        "GitHub Open Issues (log)",
        "repo-days-of-existence",
        "repo-agg-info-entropy",
    ]
)

fig = px.density_heatmap(
    df_plot,
    x="repo-days-of-existence",
    y="repo-agg-info-entropy",
    z="GitHub Open Issues (log)",  # value to aggregate per bin
    histfunc="avg",  # mean log open issues per bin
    nbinsx=60,
    nbinsy=60,
    color_continuous_scale=px.colors.sequential.thermal,
    width=1200,
    height=700,
    title="Software entropy over time for PubMed GitHub repositories (binned density)",
)

fig.update_layout(
    font=dict(size=13),
    title={"yref": "container", "y": 0.9, "yanchor": "bottom"},
    xaxis_title="Project duration (days)",
    yaxis_title="Software entropy",
)

fig.write_image(image_file := "images/software-information-entropy-open-issues.png")
display(Image(filename=image_file))
```



Checks index

This is an index of all checks in the Almanack metrics catalog.

- repo-includes-readme (SGA-GL-0001)
- repo-includes-contributing (SGA-GL-0002)
- repo-includes-code-of-conduct (SGA-GL-0003)
- repo-includes-license (SGA-GL-0004)
- repo-is-citable (SGA-GL-0005)
- repo-default-branch-not-master (SGA-GL-0006)
- repo-includes-common-docs (SGA-GL-0007)
- repo-uses-issues (SGA-GL-0015)
- repo-pull-requests-enabled (SGA-GL-0017)
- repo-doi-valid-format (SGA-GL-0025)
- repo-check-notebook-exec-order (SGA-GL-0030)

Garden Circle

The garden circle is a section of the Software Garden Almanack associated with contributions, planning, and internal maintenance of the project.

Contributing

First of all, thank you for contributing to the Software Gardening Almanack! 🎉 💯 🪴 We're so grateful for the kindness of your effort applied to grow this project into the best that it can be.

This document contains guidelines on how to most effectively contribute to this project.

If you would like clarifications, please feel free to ask any questions or for help. We suggest asking for help from GitHub where you may need it (for example, in a GitHub issue or a pull request) by "at (@) mentioning" a Software Gardening Almanack maintainer (for example, `@username`).

Code of conduct

Our Code of Conduct (CoC) policy (located here) governs this project. By participating in this project, we expect you to uphold this code. Please report unacceptable behavior by following the procedures listed under the CoC Enforcement section.

Security

Please see our Security policy (located here) for more information on security practices and recommendations associated with this project.

Quick links

- Documentation: software-gardening/almanack

- Issue tracker: software-gardening/almanack#issues

Process

Reporting bugs or suggesting enhancements

We're deeply committed to a smooth and intuitive user experience which helps people benefit from the content found within this project. This commitment requires a good relationship and open communication with our users.

We encourage you to file a GitHub issue to report bugs or propose enhancements to improve the Software Gardening Almanack.

First, figure out if your idea is already implemented by reading existing issues or pull requests! Check the issues (software-gardening/almanack#issues) and pull requests (software-gardening/almanack) to see if someone else has already documented or began implementation of your idea. If you do find your idea in an existing issue, please comment on the existing issue noting that you are also interested in the functionality. If you do not find your idea, please open a new issue and document why it would be helpful for your particular use case.

Please also provide the following specifics:

- The version of the Software Gardening Almanack you're referencing.
- Specific error messages or how the issue exhibits itself.
- Operating system (e.g. MacOS, Windows, etc.)
- Device type (e.g. laptop, phone, etc.)
- Any examples of similar capabilities

Your first code contribution

Contributing code for the first time can be a daunting task. However, in our community, we strive to be as welcoming as possible to newcomers, while ensuring sustainable software development practices.

The first thing to figure out is specifically what you're going to contribute! We describe all future work as individual GitHub issues. For first time contributors we have specifically labeled as good first issue.

If you want to contribute code that we haven't already outlined, please start a discussion in a new issue before writing any code. A discussion will clarify the new code and reduce merge time. Plus, it's possible your contribution belongs in a different code base, and we do not want to waste your time (or ours)!

Pull requests

After you've decided to contribute code and have written it up, please file a pull request. We specifically follow a forked pull request model. Please create a fork of Software Gardening Almanack repository, clone the fork, and then create a new, feature-specific branch. Once you make the necessary changes on this branch, you should file a pull request to incorporate your changes into the main The Software Gardening Almanack repository.

The content and description of your pull request are directly related to the speed at which we are able to review, approve, and merge your contribution into The Software Gardening Almanack. To ensure an efficient review process please perform the following steps:

1. Follow all instructions in the pull request template
2. Triple check that your pull request is adding *one* specific feature. Small, bite-sized pull requests move so much faster than large pull requests.
3. After submitting your pull request, ensure that your contribution passes all status checks (e.g. passes all tests)

We require pull request review and approval by at least one project maintainer in order to merge. We will do our best to review the code additions in as soon as we can. Ensuring that you follow all steps above will increase our speed and ability to review. We will check for accuracy, style, code coverage, and scope.

Git commit messages

For all commit messages, please use a short phrase that describes the specific change. For example, “Add feature to check normalization method string” is much preferred to “change code”. When appropriate, reference issues (via `#` plus number) .

Development

Overview

We write and develop the Software Gardening Almanack in Python through Jupyter Book with related environments managed by `uv`. We use Node and NPM dependencies to assist development activities (such as performing linting checks or other capabilities). We use GitHub actions for CI/CD procedures such as automated tests.

Getting started

To enable local development, perform the following steps.

1. Install Python (suggested: use `pyenv` for managing Python versions)
2. Install uv
3. Install the uv environment: `uv sync --all-groups`
4. Install Node (suggested: use `nvm` for managing Node versions)
5. Install Node environment: `npm install`
6. Install Vale dependencies: `uv run --all-groups vale sync`
7. Optional: install book PDF rendering dependencies: `uv run --all-groups playwright install --with-deps chromium`

Development Tasks

We use Poe the Poet to define common development tasks, which simplifies repeated commands. We include Poe the Poet as a Python `dev` dependency group dependency, which users access through the uv environment. Please see the `pyproject.toml` file’s `[tool.poe.tasks]` table for a list of available tasks.

For example:

```
# example of how Poe the Poet commands may be used.
uv run --all-groups poe <task_name>

# example of a task which runs jupyter-book build for the project
uv run --all-groups poe build-book
```

Linting

pre-commit automatically checks all work added to this repository. We implement these checks using GitHub Actions. Pre-commit can work alongside your local git with git-hooks After installing pre-commit within your development environment, the following command performs the same checks:

```
% pre-commit run --all-files
```

We strive to provide comments about each pre-commit check within the `.pre-commit-config.yaml` file. Comments are provided within the file to make sure we single-source documentation where possible, avoiding possible drift between the documentation and the code which runs the checks.

Automated CI/CD with GitHub Actions

We use GitHub Actions to help perform automated CI/CD as part of this project. GitHub Actions involves defining workflows through YAML files. These workflows include one or more jobs which are collections of individual processes (or steps) which run as part of a job. We define GitHub Actions work under the `.github` directory. We suggest the use of `act` to help test GitHub Actions work during development.

Releases

We utilize semantic versioning (“semver”) to distinguish between major, minor, and patch releases. We publish source code by using GitHub Releases available here. Contents of the book are distributed as both a website and PDF. We distribute a Python package through the Python Packaging Index (PyPI) available here which both includes and provides tooling for applying the book’s content.

Publishing Software Gardening Almanack releases involves several manual and automated steps. See below for an overview of how this works.

Version specifications

We follow semantic version (semver) specifications with this project through the following technologies.

- `setuptools-scm` creates version data based on git tags and commits.
- GitHub Releases automatically create git tags and source code collections available from the GitHub repository.
- `release-drafter` infers and describes changes since last release within automatically drafted GitHub Releases after pull requests are merged (draft releases are published as decided by maintainers).

Release process

1. Open a pull request and use a repository label for `release-<semver release type>` to label the pull request for visibility with `release-drafter` (for example, see almanack#43 as a reference of a semver patch update).
2. On merging the pull request for the release, a GitHub Actions workflow defined in `draft-release.yml` leveraging `release-drafter` will draft a release for maintainers.
3. The draft GitHub release will include a version tag based on the GitHub PR label applied and `release-drafter`.
4. Make modifications as necessary to the draft GitHub release, then publish the release (the draft release does not normally need additional modifications).
5. On publishing the release, another GitHub Actions workflow defined in `publish-pypi.yml` will automatically build and deploy the Python package to PyPI (utilizing the earlier modified `pyproject.toml` semantic version reference for labeling the release).
6. Each GitHub release will trigger a Zenodo GitHub integration which creates a new Zenodo record and unique DOI.

Citations

We create a unique DOI per release through the Zenodo GitHub integration. As part of this we also use a special DOI provided from Zenodo to reference the latest record on each new release. Zenodo outlines how this works within their versioning documentation.

The DOI and other citation metadata are stored within a `CITATION.cff` file. Our `CITATION.cff` is the primary source of citation information in correspondence with this project. The `CITATION.cff` file is used within a sidebar integration through the GitHub web interface to enable people to directly cite this project.

Attribution

We sourced portions of this contribution guide from `pyctyominer`. Big thanks go to the developers and contributors of that repository.

Garden Map

Our garden map is where you can learn about what features we're working on, what stage they're in, and when we expect to bring them to you. The content here follows that of a technical roadmap (providing strategic visibility on implementation details and timelines).

Ethos

We intend for the garden map to provide useful tools for us to efficiently create and maintain the Software Gardening Almanack. We practice keeping the garden map as "petal-light" as possible through built-in tools, automation, and an avoidance of analysis paralysis where possible.

We'd love your input

We welcome your input on the garden map and elements found within it! Please see the contributing guide for more information on how to participate in this process.

Active garden map(s)

The following are active garden maps for the Software Gardening Almanack. Please see below for a further description of these projects.

- **2024 BSSw Fellowship Milestones**: this project communicates activities associated with the 2024 BSSw Fellowship associated with the Software Gardening Almanack.

Sections

You can explore the garden map with different views:

- **“Canopy-view” development visibility**: We track new, existing, and previous work at a high-level using organization-wide GitHub Projects.
- **“Ground-level” development visibility**: We track repository-specific work using GitHub issues and a GitHub Project field called “status”, which we associate with each issue (indicated as “Todo”, “In progress”, “Paused”, or “Done”).
- **Major milestones**: We track major milestones using common GitHub issues labels across all repositories through their associated issues.

Visualizations

It can be helpful to visualize the garden map using data plots or other images. We use GitHub Projects insights to assist with automated creation and updates to garden map activity. Please see this project insights page for an example of these (using the insights button within any GitHub Projects for viewing others).

Acknowledgments

We drew inspiration for this content from the GitHub public roadmap. Big thanks to the original works found there.

Pavilion

We sometimes share the Software Gardening Almanack through events, blog posts, or other media which we want to share with you under this pavilion.

Poster for CU Anschutz DBMI Retreat - August 2025

We created this poster to share the Almanack with audiences attending the CU Anschutz DBMI 2025 retreat.

We also archived the work for reference here: https://doi.org/10.5281/zenodo.16943577

BSSw Blog Post Introducing the Almanack - April 2025

This blog post through the Better Scientific Software (BSSw) Blog shared our initial findings and efforts to apply Software Gardening via the Almanack book and Python package.

Please use the following link to read the article: https://bssw.io/blog_posts/growing-resilient-scientific-software-ecosystems-introducing-the-software-gardening-almanack

We also archived the work for reference here: https://doi.org/10.5281/zenodo.15330727

OSSci Monthly Call - January 2025

The Open Source Science Initiative (OSSci) Map of Open Source Science (MOSS) interest group invited us to present on the Almanack on 1/13/2025.

Please see the presentation recording or slides below as a reference to this presentation.

BSSw Blog Post on Software Gardening - August 2023

This blog post through the Better Scientific Software (BSSw) Blog shared our original vision and understanding of Software Gardening as an analogy for the development and stewardship of research software.

Please use the following link to read the article: https://bssw.io/blog_posts/long-term-software-gardening-strategies-for-cultivating-scientific-development-ecosystems

We also archived the work for reference here: https://doi.org/10.5281/zenodo.15330702

`almanack` Package API

Base

Module for interacting with Software Gardening Almanack book content through a Python package.

almanack.book.read(*chapter_name: str | None = None*)

A function for reading almanack content through a package interface.

Parameters:

chapter_name – Optional[str], default None A string which indicates the short-hand name of a chapter from the book. Short-hand names are lower-case title names read from *src/book/_toc.yml*.

Returns:

None

The outcome of this function involves printing the content of the selected chapter from the short-hand name or showing available chapter names through an exception.

Setup almanack CLI through python-fire

class **almanack.cli.AlmanackCLI**

Bases: `object`

Almanack CLI class for Google Fire

The following CLI-based commands are available (and in alignment with the methods below based on their name):

- ***almanack table <repo path>*: Provides a JSON data**
  structure which includes Almanack metric data. Always returns a 0 exit.
- ***almanack check <repo path>*: Provides a report**
  of boolean metrics which include a non-zero sustainability direction (“checks”) that are failing to inform a user whether they pass. Returns non-zero exit (1) if any checks are failing, otherwise 0.

batch(***output_path: str | None*** = *None*, ***parquet_path: str | None*** = *None*, ***repo_urls: List[str] | None*** = *None*, ***column: str*** = *'github_link'*, ***batch_size: int*** = *500*, ***max_workers: int*** = *16*, ***limit: int | None*** = *None*, ***compression: str*** = *'zstd'*, ***show_repo_progress: bool*** = *True*, ***processor: str | None*** = *None*, ***executor: str*** = *'process'*, ***split_batches: bool*** = *False*, ***collect_dataframe: bool*** = *True*, ***show_batch_progress: bool*** = *False*, ***show_errors: bool*** = *True*) → **None**

Run Almanack across many repositories defined in a parquet file or a provided list.

Example

almanack batch links.parquet results.parquet –column github_link –batch_size 1000 –max_workers 8

Parameters:

- **output_path** – Optional destination parquet for aggregated results. If omitted, results are printed as JSON.
- **parquet_path** – Parquet file containing repository URLs.
- **repo_urls** – Optional list of repository URLs to process.
- **output_path** – Destination parquet for aggregated results.
- **column** – Column name holding repository URLs.
- **batch_size** – Repositories per batch.
- **max_workers** – Parallel workers per batch.
- **limit** – Optional maximum repositories to process.
- **compression** – Parquet compression codec (default zstd).
- **show_repo_progress** – Print per-repository progress to stdout.
- **show_batch_progress** – Print per-batch progress to stdout.
- **show_errors** – Print repository-level errors to stdout.

- **processor** – Optional import path to a processor function (e.g., module:function). Defaults to Almanack processor.
- **executor** – Parallelism backend: “process” (default) or “thread”.
- **split_batches** – If True, write one parquet file per batch inside output_path (must be a directory).
- **collect_dataframe** – If False, skip retaining the combined DataFrame (avoids large in-memory data).

check(*repo_path: str*, *ignore: List[str] | None = None*, *exclude_paths: List[str] | None = None*, *verbose: bool = False*) → **None**

Check sustainability metrics and report failures.

Parameters:

- **repo_path** – The path to the repository to analyze.
- **ignore** – A list of metric IDs to ignore when running the checks.
- **exclude_paths** – Repository-relative paths or glob patterns to exclude.
- **verbose** – If True, print extra information and enable debug logging.

table(*repo_path: str*, *dest_path: str | None = None*, *ignore: List[str] | None = None*, *exclude_paths: List[str] | None = None*, *verbose: bool = False*) → **None**

Generate a table of metrics for a repository.

Parameters:

- **repo_path** – The path to the repository to analyze.
- **dest_path** – A path to send the output to.
- **ignore** – A list of metric IDs to ignore when running the checks.
- **exclude_paths** – Repository-relative paths or glob patterns to exclude.
- **verbose** – If True, print extra information and enable debug logging.

almanack.cli.trigger()

Trigger the CLI to run.

This module performs git operations

almanack.git.clone_repository(*repo_url: str*) → **Path**

Clones the GitHub repository to a temporary directory.

Parameters:

repo_url (*str*) – The URL of the GitHub repository.

Returns:

Path to the cloned repository.

Return type:

pathlib.Path

almanack.git.count_files(*tree: Tree | Blob*) → int

Counts all files (Blobs) within a Git tree, including files in subdirectories.

This function recursively traverses the provided *tree* object to count each file, represented as a *pygit2.Blob*, within the tree and any nested subdirectories.

Parameters:

tree (*Union[pygit2.Tree, pygit2.Blob]*) – The Git tree object (of type *pygit2.Tree*) to traverse and count files. The initial call should be made with the root tree of a commit.

Returns:

The total count of files (Blobs) within the tree, including nested files in subdirectories.

Return type:

int

almanack.git.detect_encoding(*blob_data: bytes*) → str

Detect the encoding of the given blob data using charset-normalizer.

Parameters:

blob_data (*bytes*) – The raw bytes of the blob to analyze.

Returns:

The best detected encoding of the blob data.

Return type:

str

Raises:

ValueError – If no encoding could be detected.

almanack.git.file_exists_in_repo(*repo: Repository*, *expected_file_name: str*, *check_extension: bool* = *False*, *extensions: list[str]* = *['.md', '.txt', '.rtf', '']*, *subdir: str | None* = *None*) → bool

Check if a file (case-insensitive and with optional extensions) exists in the latest commit of the repository.

Parameters:

- **repo** (*pygit2.Repository*) – The repository object to search in.
- **expected_file_name** (*str*) – The base file name to check (e.g., “readme”).
- **check_extension** (*bool*) – Whether to check the extension of the file or not.
- **extensions** (*list[str]*) – List of possible file extensions to check (e.g., [“.md”, “”]).
- **subdir** (*str, optional*) – Subdirectory to check within the repository tree (case-sensitive).

Returns:

True if the file exists, False otherwise.

Return type:

bool

almanack.git.find_file(*repo: Repository*, *filepath: str*, *case_insensitive: bool* = *False*, *extensions: list[str]* = *['.md', '.txt', '.rtf', '.rst', '']*) → Object | None

Locate a file in the repository by its path.

Parameters:

- **repo** (*pygit2.Repository*) – The repository object.
- **filepath** (*str*) – The path to the file within the repository.
- **case_insensitive** (*bool*) – If True, perform case-insensitive comparison.
- **extensions** (*list[str]*) – List of possible file extensions to check (e.g., [".md", ""]).

Returns:

The entry of the found file, or None if no matching file is found.

Return type:

Optional[pygit2.Object]

almanack.git.get_commits(*repo: Repository*) → List[Commit]

Retrieves the list of commits from the main branch.

Parameters:

repo (*pygit2.Repository*) – The Git repository.

Returns:

List of commits in the repository.

Return type:

List[pygit2.Commit]

almanack.git.get_edited_files(*repo: Repository*, *source_commit: Commit*, *target_commit: Commit*) → List[str]

Finds all files that have been edited, added, or deleted between two specific commits.

Parameters:

- **repo** (*pygit2.Repository*) – The Git repository.
- **source_commit** (*pygit2.Commit*) – The source commit.
- **target_commit** (*pygit2.Commit*) – The target commit.

Returns:

List of file names that have been edited, added, or deleted between the two commits.

Return type:

List[str]

almanack.git.get_loc_changed(*repo_path: Path*, *source: str*, *target: str*, *file_names: List[str]*) → Dict[str, int]

Finds the total number of code lines changed for each specified file between two commits.

Parameters:

- **repo_path** (*pathlib.Path*) – The path to the git repository.
- **source** (*str*) – The source commit hash.
- **target** (*str*) – The target commit hash.

- **file_names** (*List[str]*) – List of file names to calculate changes for.

Returns:

A dictionary where the key is the filename, and the value is the lines changed (added and removed).

Return type:

Dict[str, int]

almanack.git.get_most_recent_commits(*repo_path: Path*) → tuple[str, str]

Retrieves the two most recent commit hashes in the test repositories

Parameters:

repo_path (*pathlib.Path*) – The path to the git repository.

Returns:

Tuple containing the source and target commit hashes.

Return type:

tuple[str, str]

almanack.git.get_remote_url(*repo: Repository*) → str | None

Determines the remote URL of a git repository, if available. We use the *upstream* remote first, then *origin*, and finally any other remote. The upstream remote is preferred because it will be used for referential data lookups (such as GitHub issues, stars, etc.).

Parameters:

repo (*pygit2.Repository*) – The pygit2 repository object.

Returns:

The remote URL if found, otherwise None.

Return type:

Optional[str]

almanack.git.read_file(*repo: Repository*, *entry: Object | None* = *None*, *filepath: str | None* = *None*, *case_insensitive: bool* = *False*) → str | None

Read the content of a file from the repository.

Parameters:

- **repo** (*pygit2.Repository*) – The repository object.
- **entry** (*Optional[pygit2.Object]*) – The entry of the file to read. If not provided, filepath must be specified.
- **filepath** (*Optional[str]*) – The path to the file within the repository. Used if entry is not provided.
- **case_insensitive** (*bool*) – If True, perform case-insensitive comparison when using filepath.

Returns:

The content of the file as a string, or None if the file is not found or reading fails.

Return type:

Optional[str]

almanack.git.repo_dir_exists(*repo: Repository*, *directory_name: str*) → bool

Checks if a directory with the given name exists in the latest commit of the repository.

Parameters:

- **repo** (*pygit2.Repository*) – The repository object to search in.
- **directory_name** (*str*) – The name of the directory to look for.

Returns:

True if the directory exists, False otherwise.

Return type:

bool

almanack.git.resolve_redirects(*url: str*, *timeout: int = 10*) → str

Follow HTTP redirects until the final URL is reached.

Parameters:

- **url** (*str*) – The starting URL to check.
- **timeout** (*int, optional*) – Timeout (in seconds) for each request, by default 10.

Returns:

The last non-redirect URL.

Return type:

str

Batch processing utilities for running Almanack across many repositories.

almanack.batch_processing._nullable_dtype(*dtype: Any*) → Any

Map to nullable pandas dtypes so missing values keep schema.

almanack.batch_processing.process_repositories_batch(*repo_urls: ~typing.Sequence[str], output_path: str | ~pathlib.Path | None = None, split_batches: bool = False, collect_dataframe: bool = True, batch_size: int = 500, max_workers: int = 16, limit: int | None = None, compression: str = 'zstd', processor: ~typing.Callable[[str], dict[str, ~typing.Any]] = <function process_repo_for_almanack>, executor_cls: ~typing.Type[~concurrent.futures._base.Executor] = <class 'concurrent.futures.process.ProcessPoolExecutor'>, show_repo_progress: bool = True, show_batch_progress: bool = False, show_errors: bool = True*) → DataFrame | None

Processes repositories in batches and writes results to a single parquet file.

Parameters:

- **repo_urls** – Iterable of repository URLs to process.
- **output_path** – Optional destination parquet path for all results.
- **split_batches** – If True, write one parquet file per batch to the directory at output_path. If False (default), append all batches into a single parquet file.
- **collect_dataframe** – If False, skip retaining per-batch DataFrames and return None to reduce memory.

- **batch_size** – Number of repositories per batch.
- **max_workers** – Maximum parallel workers for each batch.
- **limit** – Optional maximum number of repositories to process.
- **compression** – Parquet compression codec (default: zstd).
- **processor** – Callable used to process a repository (default: process_repo_for_almanack).
- **executor_cls** – Executor class used for parallelism (default: ProcessPoolExecutor).
- **show_repo_progress** – Whether to print per-repository progress to stdout.
- **show_batch_progress** – Whether to print per-batch progress to stdout.
- **show_errors** – Whether to print repository-level errors to stdout.

Returns:

A DataFrame containing all processed results.

almanack.batch_processing.sanitize_for_parquet(*df: DataFrame*) → DataFrame

Cleans a DataFrame so all columns are parquet-safe.

- Expands dict columns into multiple fields.
- Converts lists into JSON strings.
- Casts generic object types into strings.

Parameters:

df – Input DataFrame with raw metrics.

Returns:

Sanitized DataFrame safe for parquet storage.

Return type:

df

Reporting

This module creates entropy reports

almanack.reporting.report.pr_report(*data: Dict[str, Any]*) → str

Returns the formatted PR-specific entropy report.

Parameters:

data (*Dict[str, Any]*) – Dictionary with the entropy data.

Returns:

Formatted GitHub markdown.

Return type:

str

almanack.reporting.report.repo_report(*data: Dict[str, Any]*) → str

Returns the formatted entropy report as a string.

Parameters:

data (*Dict[str, Any]*) – Dictionary with the entropy data.

Returns:

Formatted entropy report.

Return type:

str

Metrics

Data

This module computes data for GitHub Repositories

almanack.metrics.data._get_almanack_version() → str

Seeks the current version of almanack from package metadata.

Returns:

str

A string representing the version of almanack currently being used.

almanack.metrics.data._get_cli_entrypoints(*repo: Repository*) → List[str]

Return sorted CLI entrypoint names discovered from pyproject.toml, setup.cfg, and setup.py.

Parameters:

repo – The pygit2 Repository to read packaging files from.

Returns:

A sorted list of unique CLI command names. Returns an empty list if no entrypoints are found.

almanack.metrics.data._get_conda_python_version(*content: str*) → str | None

Return the Python version constraint declared in a conda environment YAML string.

Scans the `dependencies` list for an entry that starts with `python` followed by a version separator (`=`, `>`, `<`, `~`). Returns `None` if no Python dependency is found or the YAML cannot be parsed.

Parameters:

content – The raw string contents of a conda `environment.yml` file.

Returns:

The Python version string extracted from the dependency entry (for example, `"3.11"`), or `None` if no Python version is declared or the file cannot be parsed.

almanack.metrics.data._get_programming_extensions() → frozenset[str]

Return file extensions that belong to programming languages per GitHub Linguist.

Fetches and parses the Linguist `languages.yml` on the first call, then caches the result for the lifetime of the process. Falls back to `_FALLBACK_PROGRAMMING_EXTENSIONS` if the fetch or parse fails.

almanack.metrics.data._get_pyproject_python_version(*content: str*) → str | None

Return the declared Python version constraint from a pyproject.toml string.

Checks `tool.poetry.dependencies.python` first (Poetry convention), then `project.requires-python` (PEP 621). Returns `None` if neither is present or if the TOML cannot be parsed.

Parameters:

content – The raw string contents of a `pyproject.toml` file.

Returns:

The Python version constraint string (for example, `">=3.9"`), or `None` if no constraint is declared or the file cannot be parsed.

almanack.metrics.data._get_python_environment_data(*repo: Repository*) → Dict[str, Any]

Detect Python environment and dependency management tools in the repository.

Scans common Python packaging and environment configuration files to identify which tools are in use and what Python versions are declared. This function is intended for Python-primary repositories; call sites should guard invocation with a primary-language check.

Parameters:

repo – The pygit2 Repository to scan for environment configuration files.

Returns:

- `environment_managers`: sorted list of detected environment manager names (for example, `["conda", "poetry"]`), or `None` if none are found.
- `has_managed_environment`: `True` if at least one environment manager is detected, `False` otherwise.
- `dependency_managers`: sorted list of detected dependency manager names (for example, `["pip"]`), or `None` if none are found.
- `has_declared_dependencies`: `True` if at least one dependency manager is detected, `False` otherwise.
- `declared_python_versions`: sorted list of declared Python version strings, or `None` if none are found.

Return type:

A dictionary with the following keys

almanack.metrics.data._get_repository_languages_data(*remote_repo_data: Dict[str, Any]*, *remote_url: str | None*) → Dict[str, int]

Return repository language line counts from remote metadata or GitHub.

Parameters:

- **remote_repo_data** – Metadata dict from a hosting platform or ecosyste.ms mirror. May contain a `languages_lines`, `languages_loc`, or `languages` key with per-language counts.
- **remote_url** – Remote URL of the repository, used to fall back to the GitHub languages API when hosting metadata is unavailable.

Returns:

A dictionary mapping programming language name to an approximate line or byte count. Returns an empty dict if no language data is found.

almanack.metrics.data._get_software_description(*repo: Repository*, *remote_repo_data: Dict[str, Any] | None*, *readme_exists: bool*, *readme_file: Object | None*) → str | None

Return the best available plain-text description for the repository.

The function uses the following priority order: 1. The description field from remote hosting metadata (e.g. ecosyste.ms / GitHub). 2. The `abstract` field from a `CITATION.cff` file in the repo root. 3. The first non-badge paragraph of the README.

Returns the first non-empty candidate, or `None` if the repo doesn't include any priority entry.

Parameters:

- **repo** – The pygit2 Repository to read local files from.
- **remote_repo_data** – Metadata dict from a hosting platform or ecosyste.ms mirror, used to retrieve the remote description field.
- **readme_exists** – Whether a README file was detected in the repository.
- **readme_file** – The pygit2 object for the README file, used to read its contents when falling back to the first paragraph.

Returns:

The best available plain-text description string, or `None` if no description could be found.

almanack.metrics.data._normalize_exclude_paths(*exclude_paths: str | List[str] | Tuple[str, ...] | None*) → List[str] | None

Normalize exclude path inputs into a list of strings.

Parameters:

exclude_paths – A string, list, or tuple of exclude paths. Accepts comma-separated strings and strips surrounding quotes.

Returns:

A list of normalized exclude paths, or None if no paths are provided.

almanack.metrics.data._parse_setup_py_console_scripts(*content: str*) → set[str]

Extract console_scripts command names from a setup.py string.

Parses the content as a Python Abstract Syntax Tree (AST) and looks for a `setup()` or `setuptools.setup()` call whose `entry_points` keyword argument contains a `console_scripts` list. Returns an empty set if the file cannot be parsed or contains no matching entries.

Parameters:

content – The raw string contents of a `setup.py` file.

Returns:

A set of CLI command names declared in `console_scripts`. Returns an empty set if none are found or the file cannot be parsed.

almanack.metrics.data._walk_tree_measure_size_of_noncode_files(*tree: Tree | Blob*, *repo: Repository*, *prefix: str = ''*) → Dict[str, int]

Recursively walk a git tree and count lines per non-code file extension.

The function iterates through each blob (file) in the tree, and counts new lines for non-code files. Specifically, the function will skip a file if the extension appears in `_get_programming_extensions()` or if it lives inside a `.git/` directory. The function counts non-code, text-based files by newline and counts binary files based on byte length. Lastly, the function computes a sum per file extension and writes into a `{extension: line_count}` dict where files missing an extension use the key `"<no_ext>"`.

Parameters:

- **tree** – A pygit2 Tree or Blob to walk. Top-level callers pass the root tree; recursive calls pass subtrees or blobs.
- **repo** – The pygit2 Repository used to dereference object IDs when traversing subtrees.
- **prefix** – Relative file path accumulated during recursion. Defaults to an empty string for the root call.

Returns:

A dictionary mapping each non-code file extension (for example, `.md`, `.csv`) to its approximate line or byte count. Files without an extension are keyed as `"<no_ext>"`.

almanack.metrics.data.compute_almanack_score(*almanack_table: List[Dict[str, int | float | bool]]*) → Dict[str, int | float]

Computes an Almanack score by counting boolean Almanack table metrics to provide a quick summary of software sustainability.

Parameters:

almanack_table (*List[Dict[str, Union[int, float, bool]]]*) – A list of dictionaries containing metrics. Each dictionary must have a “result” key with a value that is an int, float, or bool. A “sustainability_correlation” key is included for values to specify the relationship to sustainability: - 1 (positive correlation) - 0 (no correlation) - -1 (negative correlation)

Returns:

Dictionary of length three, including the following: 1) number of Almanack boolean metrics that passed (numerator), 2) number of total Almanack boolean metrics considered (denominator), and 3) a score that represents how likely the repository will be maintained over time based (numerator / denominator).

Return type:

Dict[str, Union[int, float]]

almanack.metrics.data.compute_pr_data(*repo_path: str*, *pr_branch: str*, *main_branch: str*) → Dict[str, Any]

Computes entropy data for a PR compared to the main branch.

Parameters:

- **repo_path** (*str*) – The local path to the Git repository.
- **pr_branch** (*str*) – The branch name for the PR.
- **main_branch** (*str*) – The branch name for the main branch.

Returns:

A dictionary containing the following key-value pairs:

- ”pr_branch”: The PR branch being analyzed.
- ”main_branch”: The main branch being compared.
- ”total_entropy_introduced”: The total entropy introduced by the PR.
- ”number_of_files_changed”: The number of files changed in the PR.
- ”entropy_per_file”: A dictionary of entropy values for each changed file.
- ”commits”: A tuple containing the most recent commits on the PR and main branches.

Return type:
dict

almanack.metrics.data.compute_repo_data(*repo_path: str*, *exclude_paths: List[str] | None* = *None*, *required_metric_names: set[str] | None* = *None*) → Dict[str, Any]

Compute comprehensive data for a GitHub repository.

Parameters:
- **repo_path** – The local path to the Git repository.
- **exclude_paths** – Repository-relative paths or glob patterns to exclude from checks.
- **required_metric_names** – Optional set of metric names that will be used by downstream reporting. If provided, expensive data-collection blocks are skipped unless one of their dependent metrics is requested.

Returns:
A dictionary containing data key-pairs.

almanack.metrics.data.days_of_development(*repo: Repository*) → float

Parameters:
repo (*pygit2.Repository*) – Path to the git repository.

Returns:
The average number of commits per day over the period of time.

Return type:
float

almanack.metrics.data.gather_failed_almanack_metric_checks(*repo_path: str*, *ignore: List[str] | None* = *None*, *exclude_paths: List[str] | None* = *None*) → List[Dict[str, Any]]

Gather checks on the repository metrics and return a list of failed checks.

Parameters:
- **repo_path** – The file path to the repository which will have metrics calculated and includes boolean checks.
- **ignore** – A list of metric IDs to ignore when running the checks.
- **exclude_paths** – Repository-relative paths or glob patterns to exclude from checks.

Returns:
A list of dictionaries containing the metrics and their associated results. Each dictionary includes the name, id, and guidance on how to fix each failed check. The dictionary also includes data about the

almanack score for use in summarizing the results.

almanack.metrics.data.get_github_build_metrics(*repo_url: str*, *branch: str* = *'main'*, *max_runs: int* = *100*, *github_api_endpoint: str* = *'https://api.github.com/repos'*) → dict

Fetches the success ratio of the latest GitHub Actions build runs for a specified branch.

Parameters:

- **repo_url** (*str*) – The full URL of the repository (e.g., ‘ software-gardening/almanack’).
- **branch** (*str*) – The branch to filter for the workflow runs (default: “main”).
- **max_runs** (*int*) – The maximum number of latest workflow runs to analyze.
- **github_api_endpoint** (*str*) – Base API endpoint for GitHub repositories.

Returns:

The success ratio and details of the analyzed workflow runs.

Return type:

dict

almanack.metrics.data.get_table(*repo_path: str*, *ignore: List[str] | None* = *None*, *exclude_paths: List[str] | None* = *None*) → List[Dict[str, Any]]

Gather metrics on a repository and return the results in a structured format.

This function reads a metrics table from a predefined YAML file, computes relevant data from the specified repository, and associates the computed results with the metrics defined in the metrics table. If an error occurs during data computation, an exception is raised.

Parameters:

- **repo_path** – The file path to the repository for which the Almanack runs metrics.
- **ignore** – A list of metric IDs to ignore when running the checks.
- **exclude_paths** – Repository-relative paths or glob patterns to exclude from checks.

Returns:

A list of dictionaries containing the metrics and their associated results. Each dictionary includes the original metrics data along with the computed result under the key “result”.

Raises:

ReferenceError – If there is an error encountered while processing the data, providing context in the error message.

almanack.metrics.data.is_conda_environment_yaml(*content: str*) → bool

Return True if *content* looks like a conda environment YAML file.

A conda environment file is a YAML document whose top-level value is a mapping that contains at least one of the keys `name`, `channels`, or `dependencies`. This heuristic avoids false positives from arbitrary YAML files that happen to share the same filename.

Parameters:

content – The raw string contents of a YAML file to inspect.

Returns:

`True` if the content matches the conda environment YAML heuristic, `False` otherwise.

almanack.metrics.data.measure_coverage(*repo: Repository*, *primary_language: str | None*) → Dict[str, Any] | None

Measures code coverage for a given repository.

Parameters:

- **repo** (*pygit2.Repository*) – The pygit2 repository object to analyze.
- **primary_language** (*Optional[str]*) – The primary programming language of the repository.

Returns:

Code coverage data or an empty dictionary if unable to find code coverage data.

Return type:

Optional[dict[str,Any]]

almanack.metrics.data.parse_python_coverage_data(*repo: Repository*) → Dict[str, Any] | None

Parses coverage.py data from recognized formats such as JSON, XML, or LCOV. See here for more information: https://coverage.readthedocs.io/en/latest/cmd.html#cmd-report

Parameters:

repo (*pygit2.Repository*) – The pygit2 repository object containing code.

Returns:

A dictionary with standardized code coverage data or an empty dict if no data is found.

Return type:

Optional[Dict[str, Any]]

almanack.metrics.data.process_repo_for_almanack(*repo_url: str*, *exclude_paths: List[str] | None* = *None*) → Dict[str, Any]

Process a GitHub repository URL into a flat dictionary of Almanack metrics.

Parameters:

- **repo_url** – The GitHub repository URL.
- **exclude_paths** – Repository-relative paths or glob patterns to exclude from checks.

Returns:

Flattened metrics for the repository, including sustainability checks. If the processing fails, returns an error entry with the repository URL.

almanack.metrics.data.process_repo_for_analysis(*repo_url: str*) → Tuple[float | None, str | None, str | None, int | None]

Processes GitHub repository URL's to calculate entropy and other metadata. This is used to prepare data for analysis, particularly for the seedbank notebook that process PUBMED repositories.

Parameters:

repo_url (*str*) – The URL of the GitHub repository.

Returns:

A tuple containing the normalized total entropy, the date of the first commit,

the date of the most recent commit, and the total time of existence in days.

Return type:

tuple

almanack.metrics.data.table_to_wide(*table_rows: list[dict]*) → Dict[str, Any]

Transpose Almanack table (name->result), compute checks summary, flatten nested. *repo-file-info-entropy* and *repo-file-history-complexity-decay* are omitted from the wide output because they are large, file-level metrics that are out of scope for this flattened summary representation.

Parameters:

table_rows (*list[dict]*) – The Almanack metrics table as a list of dictionaries, each containing metric metadata and a "result" field.

Returns:

A flattened dictionary mapping metric names to their results, including computed summary fields:

- "checks_total": total number of sustainability-related checks
- "checks_passed": number of checks passed
- "checks_pct": percentage of checks passed

Return type:

dict

Remote

This module focuses on remote API requests and related aspects.

almanack.metrics.remote.get_api_data(*api_endpoint: str = 'https://repos.ecosyste.ms/api/v1/repositories/lookup', params: Dict[str, str] | None = None*) → dict

Get data from an API based on the remote URL, with retry logic for GitHub rate limiting.

Parameters:

- **api_endpoint** (*str*) – The HTTP API endpoint to use for the request.
- **params** (*Optional[Dict[str, str]]*) – Additional query parameters to include in the GET request.

Returns:

The JSON response from the API as a dictionary.

Return type:

dict

Raises:

requests.RequestException – If the API call fails for reasons other than rate limiting.

almanack.metrics.remote.request_with_backoff(*method: str*, *url: str*, ***, *headers: Dict[str, str] | None = None*, *params: Dict[str, str] | None = None*, *timeout: int = 30*, *allow_redirects: bool | None = None*, *max_retries: int = 5*, *base_backoff: float = 1.0*, *backoff_multiplier: float = 2.0*, *retry_statuses: Set[int] | None = None*) → Response | None

Perform an HTTP request with retry using a backoff for transient failures.

Parameters:

- **method** (*str*) – The HTTP method to use (e.g., “GET”, “HEAD”).
- **url** (*str*) – The URL to request.
- **headers** (*Optional[Dict[str, str]]*) – Optional HTTP headers.
- **params** (*Optional[Dict[str, str]]*) – Optional query parameters.
- **timeout** (*int*) – Request timeout in seconds.
- **allow_redirects** (*Optional[bool]*) – Whether to follow redirects.
- **max_retries** (*int*) – Maximum number of attempts before giving up.
- **base_backoff** (*float*) – Base backoff duration in seconds.
- **backoff_multiplier** (*float*) – Multiplier for exponential backoff growth.
- **retry_statuses** (*Optional[Set[int]]*) – HTTP status codes to retry.

Returns:

The response on success, or None on failure.

Return type:

Optional[requests.Response]

Entropy

Calculate software change entropy and decay-weighted history complexity.

class **almanack.metrics.entropy.calculate_entropy.HistoryComplexityConfig**(*decay_factor: float = 10.0*, *quiet_time_seconds: int = 3600*)

Bases: `object`

Configuration values for the decay-weighted history complexity metric.

decay_factor

Time scale (in hours) for exponential down-weighting of older burst periods. Larger values make older periods lose influence more slowly; smaller values make them fade faster.

Type:

float

quiet_time_seconds

Maximum allowed time gap between adjacent commit events in the same burst period. If the gap is larger than this threshold, a new burst period starts.

Type:

int

decay_factor: *float = 10.0*

quiet_time_seconds: *int = 3600*

almanack.metrics.entropy.calculate_entropy._collect_period_file_changes(*repo: Repository*, *source_commit: Commit*, *target_commit: Commit*, *tracked_files: set[str]*, *quiet_time_seconds: int = 3600*) → **list[tuple[int, dict[str, int]]]**

Collect per-period tracked-file change totals separated by quiet windows.

This function walks commits from *target_commit* backwards to *source_commit*, extracts tracked-file line changes from each commit, and then groups these commit events into burst periods. A commit event is one commit timestamp plus tracked-file counts of lines added and deleted. A new period starts when the elapsed time between consecutive commit events is greater than *quiet_time_seconds*.

Parameters:

- **repo** – Open repository used to read commit history and diffs.
- **source_commit** – Commit that marks the start boundary (exclusive).
- **target_commit** – Commit that marks the end boundary (inclusive).
- **tracked_files** – File paths that should contribute to change totals.
- **quiet_time_seconds** – Threshold separating consecutive burst periods.

Returns:

Time-ordered list of periods. Each item contains the period end time (Unix timestamp) and file-level counts of lines added plus deleted for that period.

almanack.metrics.entropy.calculate_entropy._group_commit_events_by_quiet_window(*commit_events: list[tuple[int, dict[str, int]]]*, *quiet_time_seconds: int*) → **list[tuple[int, dict[str, int]]]**

Group commit events into burst periods separated by quiet windows.

Parameters:

- **commit_events** – Sequence of commit events, where each event is *(event_time, file_changes)*. *event_time* is a commit timestamp, and *file_changes* maps tracked file paths to counts of lines added plus deleted from that commit.
- **quiet_time_seconds** – Threshold that starts a new period when exceeded.

Returns:

Time-ordered periods with end timestamp and aggregated file changes.

almanack.metrics.entropy.calculate_entropy._resolve_commit(*repo: Repository*, *commit_ref: str | Commit*) → **Commit**

Convert a commit input into a concrete *pygit2.Commit*.

Parameters:

- **repo** – Open repository used to look up commit reference strings.

- **commit_ref** – Existing commit object or rev-parse compatible commit string.

Returns:

Commit object for *commit_ref*.

almanack.metrics.entropy.calculate_entropy.calculate_aggregate_entropy(*repo_path: Path*, *source_commit: str | Commit*, *target_commit: str | Commit*, *file_names: List[str]*) → float

Calculate mean normalized entropy across the provided files.

Parameters:

- **repo_path** – Path to the local Git repository.
- **source_commit** – Commit object or reference string for range start.
- **target_commit** – Commit object or reference string for range end.
- **file_names** – Repository-relative file paths to include in the calculation.

Returns:

Mean of per-file normalized entropy values, or *0.0* for no files.

References

Hassan, A. E. (2009). Predicting faults using the complexity of code changes. 2009 IEEE 31st International Conference on Software Engineering, 78-88. https://doi.org/10.1109/ICSE.2009.5070510

almanack.metrics.entropy.calculate_entropy.calculate_aggregate_history_complexity_with_decay(*repo_path: Path*, *source_commit: str | Commit*, *target_commit: str | Commit*, *file_names: List[str]*, *config: HistoryComplexityConfig = HistoryComplexityConfig(decay_factor=10.0, quiet_time_seconds=3600)*) → float

Calculate the mean decay-weighted history complexity across files.

Parameters:

- **repo_path** – Path to the local Git repository.
- **source_commit** – Commit object or reference string for range start.
- **target_commit** – Commit object or reference string for range end.
- **file_names** – Repository-relative file paths to include in the mean.
- **config** – Decay and period-grouping configuration.

Returns:

Mean file-level history complexity score, or *0.0* for no files.

References

Hassan, A. E. (2009). Predicting faults using the complexity of code changes. 2009 IEEE 31st International Conference on Software Engineering, 78-88. https://doi.org/10.1109/ICSE.2009.5070510

almanack.metrics.entropy.calculate_entropy.calculate_history_complexity_with_decay(*repo_path: Path*, *source_commit: str | Commit*, *target_commit: str | Commit*, *file_names: list[str]*,

config: ***HistoryComplexityConfig*** *= HistoryComplexityConfig(decay_factor=10.0, quiet_time_seconds=3600)*) → **dict[str, float]**

Calculate decay-weighted history complexity for each tracked file.

For each burst period, this computes each file's Shannon entropy contribution *-(p_i * log2(p_i))* from file-level changed-line probabilities, then applies exponential decay by period age.

Parameters:

- **repo_path** – Path to the local Git repository.
- **source_commit** – Commit object or reference string for range start.
- **target_commit** – Commit object or reference string for range end.
- **file_names** – Repository-relative file paths to score.
- **config** – Decay and period-grouping configuration.

Returns:

Mapping of file path to decay-weighted history complexity score.

Raises:

ValueError – If *config.decay_factor* is less than or equal to zero.

References

Hassan, A. E. (2009). Predicting faults using the complexity of code changes. 2009 IEEE 31st International Conference on Software Engineering, 78-88. https://doi.org/10.1109/ICSE.2009.5070510

almanack.metrics.entropy.calculate_entropy.calculate_normalized_entropy(*repo_path: Path, source_commit: str | Commit, target_commit: str | Commit, file_names: list[str]*) → dict[str, float]

Calculate per-file normalized Shannon entropy for changed lines.

The function computes entropy using per-file change probabilities between *source_commit* and *target_commit*, where each probability is: *changed_lines_in_file / total_changed_lines_across_files*.

Parameters:

- **repo_path** – Path to the local Git repository.
- **source_commit** – Commit object or reference string for range start.
- **target_commit** – Commit object or reference string for range end.
- **file_names** – Repository-relative file paths to include in the calculation.

Returns:

Mapping of file path to that file's entropy contribution.

References

Hassan, A. E. (2009). Predicting faults using the complexity of code changes. 2009 IEEE 31st International Conference on Software Engineering, 78-88. https://doi.org/10.1109/ICSE.2009.5070510

This module procesess GitHub data

almanack.metrics.entropy.processing_repositories.process_pr_entropy(*repo_path: str*, *pr_branch: str*, *main_branch: str*) → str

Processes GitHub PR data to calculate a report comparing the PR branch to the main branch.

Parameters:

- **repo_path** (*str*) – The local path to the Git repository.
- **pr_branch** (*str*) – The branch name of the PR.
- **main_branch** (*str*) – The branch name for the main branch.

Returns:

A JSON string containing report data.

Return type:

str

Raises:

FileNotFoundError – If the specified directory does not contain a valid Git repository.

almanack.metrics.entropy.processing_repositories.process_repo_entropy(*repo_path: str*) → str

Processes GitHub repository data to calculate a report.

Parameters:

repo_path (*str*) – The local path to the Git repository.

Returns:

A JSON string containing the repository data and entropy metrics.

Return type:

str

Raises:

FileNotFoundError – If the specified directory does not contain a valid Git repository.

Garden Lattice

Understanding

This module focuses on the Almanack's Garden Lattice materials which encompass aspects of human understanding.

almanack.metrics.garden_lattice.understanding.includes_common_docs(*repo: Repository*) → bool

Check whether the repo includes common documentation files and directories associated with building docsites.

Parameters:

repo (*pygit2.Repository*) – The repository object.

Returns:

True if any common documentation files are found, False otherwise.

Return type:

bool

Connectedness

Module for Almanack metrics covering human connection and engagement in software projects such as contributor activity, collaboration frequency.

almanack.metrics.garden_lattice.connectedness._build_openalex_doi_metrics(*openalex_result: Dict[str, Any]*) → Dict[str, Any]

Build DOI-linked OpenAlex metrics for storage in citation results.

almanack.metrics.garden_lattice.connectedness._extract_doi_from_citation_data(*citation_data: Dict[str, Any]*) → str | None

Extract a DOI value from parsed CITATION.cff payload.

almanack.metrics.garden_lattice.connectedness._extract_funder_key(*funder: Any*) → str | None

Extract a stable funder key from known OpenAlex funder shapes.

almanack.metrics.garden_lattice.connectedness._funding_amount_to_usd(*amount: Any*, *currency: str | None*) → float | None

Convert a funding amount to USD; assume USD when currency is unavailable.

almanack.metrics.garden_lattice.connectedness._get_currency_converter() → CurrencyConverter

Return a cached currency converter for funding amount normalization.

almanack.metrics.garden_lattice.connectedness._get_work_funding_records(*work_data: Dict[str, Any]*) → List[Dict[str, Any]]

Return funding records from OpenAlex work payloads.

Supports both modern *awards* and legacy *grants* fields. In Almanack outputs, these are normalized as “funding records”.

almanack.metrics.garden_lattice.connectedness._summarize_openalex_funding(*work_data: Dict[str, Any]*) → Dict[str, Any]

Summarize OpenAlex funding payloads for one work record.

almanack.metrics.garden_lattice.connectedness.count_unique_contributors(*repo: Repository*, *since: datetime | None* = *None*) → int

Counts the number of unique contributors to a repository.

If a *since* datetime is provided, counts contributors who made commits after the specified datetime. Otherwise, counts all contributors.

Parameters:

- **repo** (*pygit2.Repository*) – The repository to analyze.
- **since** (*Optional[datetime]*) – The cutoff datetime. Only contributions after this datetime are counted. If None, all contributions are considered.

Returns:

The number of unique contributors.

Return type:

int

almanack.metrics.garden_lattice.connectedness.default_branch_is_not_master(*repo: Repository*) → bool

Checks if the default branch of the specified repository is “master”.

Parameters:

repo (*Repository*) – A pygit2.Repository object representing the Git repository.

Returns:

True if the default branch is “master”, False otherwise.

Return type:

bool

almanack.metrics.garden_lattice.connectedness.detect_social_media_links(*content: str*) → Dict[str, List[str]]

Analyzes README.md content to identify social media links.

Parameters:

readme_content (*str*) – The content of the README.md file as a string.

Returns:

A dictionary containing social media details discovered from readme.md content.

Return type:

Dict[str, List[str]]

almanack.metrics.garden_lattice.connectedness.find_doi_citation_data(*repo: Repository*) → Dict[str, Any]

Find and validate DOI information from a CITATION.cff file in a repository.

This function searches for a *CITATION.cff* file in the provided repository, extracts the DOI (if available), validates its format, checks its resolvability via HTTP, and performs an exact DOI lookup on the OpenAlex API.

Parameters:

repo (*pygit2.Repository*) – The repository object to search for the CITATION.cff file.

Returns:

A dictionary containing DOI-related information and metadata.

Return type:

Dict[str, Any]

almanack.metrics.garden_lattice.connectedness.find_openalex_citing_works_funding(*openalex_work_id: str | None*, *max_references: int* = *25*) → Dict[str, Any]

Find funding signals from OpenAlex works that cite the repository work.

In this context, “sampled” means the subset of citing works returned by this query, limited by *max_references* and sorted by *cited_by_count*. OpenAlex *awards* (and legacy *grants*) are normalized as “funding records”.

Parameters:

- **openalex_work_id** – OpenAlex work identifier for the project’s DOI-linked work.
- **max_references** – Maximum number of citing works to query from OpenAlex.

Returns:

Dictionary with sampled citing-work funding aggregates and references.

almanack.metrics.garden_lattice.connectedness.find_software_mentions_openalex(*repo: Repository*, *remote_url: str | None*, *max_references: int* = *10*) → Dict[str, Any]

Find OpenAlex works that mention a repository by software/project name.

Parameters:

- **repo** – Repository used for software name discovery when no remote URL is available.
- **remote_url** – Remote repository URL used to derive the project name.
- **max_references** – Maximum number of matching works to include in results.

Returns:

Dictionary containing the query name, aggregate mention count, and a minimal list of matching works from OpenAlex.

almanack.metrics.garden_lattice.connectedness.is_citable(*repo: Repository*) → bool

Check if the given repository is citable.

A repository is considered citable if it contains a CITATION.cff or CITATION.bib file, or if the README.md file contains a citation section indicated by “## Citation” or “## Citing”.

Parameters:

repo (*pygit2.Repository*) – The repository to check for citation files.

Returns:

True if the repository is citable, False otherwise.

Return type:

bool

Practicality

This module focuses on the Almanack's Garden Lattice materials which involve how people can apply software in practice.

almanack.metrics.garden_lattice.practicality.count_repo_tags(*repo: Repository*, *since: datetime | None* = *None*) → int

Counts the number of tags in a pygit2 repository.

If a *since* datetime is provided, counts only tags associated with commits made after the specified datetime. Otherwise, counts all tags in the repository.

Parameters:

- **repo** (*pygit2.Repository*) – The repository to analyze.
- **since** (*Optional[datetime]*) – The cutoff datetime. Only tags for commits after this datetime are counted. If None, all tags are counted.

Returns:

The number of tags in the repository that meet the criteria.

Return type:

int

almanack.metrics.garden_lattice.practicality.get_ecosystems_package_metrics(*repo_url: str*) → Dict[str, Any]

Fetches package data from the ecosyste.ms API and calculates metrics about the number of unique ecosystems, total version counts, and the list of ecosystem names.

Parameters:

repo_url (*str*) – The repository URL of the package to query.

Returns:

A dictionary containing information about packages related to the repository.

Return type:

Dict[str, Any]